

Machine Learning-Based Preconditioners for Dirac Operator Inversion in Lattice QCD

Ole Wolsing Kaa (olkaa21@student.sdu.dk)

Master Thesis
IMADA

Supervisor:
Benjamin Jäger



Abstract

I denne specialeafhandling ser vi på problemet med at finde en approksimation af vektoren x fra det lineære ligningssystem $Ax = b$, med hovedformålet at forkorte køretiden af den iterative metode conjugate gradient. Det gøres ved at transformere systemet med en forbedningsmatrix. Vores mål er at finde ud af, om det er muligt at lære denne matrix således, at den kan reducere køretiden for et generelt sæt af ukendte lineære systemer. Vores interesse for dette kommer fra studiet af kvantekromodynamik (QCD) med modellen Gitter-QCD, hvor den beregningsmæssige dominerende del af modellen er applikationen af den inverse Dirac-operator, D . Konvergeringsraten af conjugate gradient for fejlen i approksimationen af x kan blive afgrænset af betingelsesnummeret af matrixen, $\kappa(A)$. Derfor: hvis det lineære system bliver transformeret af forbedningsmatrixen således at betingelsesnummeret reduceres, så forkortes køretiden af conjugate gradient.

Undersøgelsen af læringen af forbedningsmatrixen bestod af to kategorier: strukturen af matrixen og læringsalgoritmen. Strukturen af forbedningsmatrixen startede som en øvre bidiagonalmatrix, med en identitet hoveddiagonal og en partitioneret superdiagonal med en koefficient for hver partitionering. Læringsalgoritmen virker ved at ændre en af koefficienterne i superdiagonalen, hvorefter den evaluerer med conjugate gradient for at bestemme om ændringen reducerer antallet af iterationer yderligere.

For at bestemme hvornår en ændring af forbedningsmatrixen er en forbedring, testede vi læringsalgoritmen med tre forskellige forbedringsfunktioner: MEAN-forbedringsfunktionen, som sammenligner gennemsnittet af iterationstallene, MEDIAN-forbedringsfunktionen, som sammenligner medianen af iterationstallene og SIGN-forbedringsfunktionen, som sammenligner de individuelle lineære systemers iterationstal. Resultaterne fra evalueringen af de lærte forbedningsmatrixer viste, at SIGN-forbedringsfunktionen frembragte den bedste forbedningsmatrix i læringen. Men i en sammenligning med en kendt forbedningsmatrix, Jacobi forbedning, der bruger hoveddiagonalen af matrixen fra det lineære system som forbedningsmatrix, var iterationstallene af de forbedrede lineære systemer noget større og havde dermed en længere køretid. Derfor ændrede vi hoveddiagonalen af vores forbedningsmatrix fra identitetsmatrixen til den inverse af hoveddiagonalen af det lineære systems matrix. Her blev læringen og evalueringen noget bedre end vores tidligere forbedningsmatrixer, men dog kun med en marginal forskel til Jacobi forbedning.

De næste strukturelle ændringer, vi testede, involverede superdiagonalen: mængden af partitioneringer og antallet af ikke-nul superdiagonaler. Resultaterne af disse læringstests viste at ændringerne af superdiagonalerne ikke medførte en forbedring af forbedningsmatrixen. Men vi kunne dog konkludere, at kun at bruge den første superdiagonal og kun én koefficient til hele superdiagonalen gav den hurtigste læring af forbedningsmatrixen, uden at den blev forringet.

Den sidste store ændring af forbedningsmatrixen var at tilføje en ikke-nul subdiagonal. Dette betyder at strukturen af forbedningsmatrixen nu er en tridiagonalmatrix med en hoveddiagonal som Jacobi forbedning og en partitioneret sub- og superdiagonal. Evalueringen af de lærte forbedningsmatrixer med den nye struktur viste en reduktion af gennemsnits iterationstallet på to gange så meget, som Jacobi forbedning reducerer. Efter undersøgelsen af den nye forbedningsmatrix, er vi kommet frem til, at den tridiagonale struktur med én koefficient i både sub- og superdiagonalerne giver de bedste resultater i evalueringen af de lærte forbedningsmatrixer. Vi kan hermed konkludere, at det er muligt at lære en generel forbedningsmatrix, der kan reducere antallet af iterationer, som en iterativ metode har brug for, for at finde en approksimation til et ukendt lineært ligningssystem.

Contents

Contents	1
1 Introduction	2
2 Iterative methods for linear systems	3
2.1 Conjugate Gradient	3
2.1.1 Convergece	5
2.1.2 Preconditioning	8
2.2 CG Variants	9
2.2.1 Normal Equation	9
2.2.2 Bi-Conjugate Gradient	9
2.2.3 Conjugate Gradient Squared	10
2.2.4 Bi-Conjugate Gradient Stabilized	11
2.2.5 Comparison	12
3 Exploration of learning preconditioners	13
3.1 Structure of the preconditioner	13
3.2 Initial learning algorithm	15
3.2.1 Better exploration of the preconditioner	17
3.3 Discrete laplace operator	19
3.4 Improvement functions	23
3.4.1 MEAN improvement function	23
3.4.2 MEDIAN improvement function	23
3.4.3 SIGN improvement function	23
3.4.4 Comparison with Jacobi preconditioning	27
3.5 Shifted Jacobi preconditioner	29
3.5.1 Plateau in learning	31
3.6 Allowing disimprovement	32
3.7 Number of partitions	34
3.7.1 Fewer	36
3.8 Number of diagonals	37
3.9 Super- and subdiagonals	39
3.9.1 Number of partitions	40
3.9.2 Symmetry	42
3.10 Testing the algorithm on larger systems	43
4 Conclusion	46
5 Bibliography	47
6 Appendix	47

1 Introduction

The problem of matrix inversion for linear systems, is a problem that arises in many fields of numerical study. Our interest comes from the study of Quantum Chromodynamics with the model Lattice QCD. Where matrix inversion comes from the need of the application of the inverse of the Dirac operator, D , onto a vector. Which can be done with the method conjugate gradients. In exact arithmetic conjugate gradients uses a maximum number of iterations equal to the dimension of the system it is used on [8] as the approximation is created from a linear combination of orthogonal vectors.

Because the numerical studies are not done in exact arithmetic. A general convergence of conjugate gradients can be derived from the error of the approximation such that it is bounded by the condition number of the matrix from the linear system, $\kappa(A)$. Meaning that system with larger condition number have a slower convergence than a system with smaller condition number. Therefore when using method such as conjugate gradient and its variants, if the linear system is transformed with a preconditioner such that the new systems condition number is much smaller and also therefore has a faster rate of convergence.

Construction of a good preconditioner is difficult as it needs to decrease the iteration count enough such that the extra computational cost of using a preconditioner does not increase the total running time of the linear solver. A good preconditioner is also often unique to the system. Therefore if there is the need to precondition many slightly different systems, a unique preconditioner has to be constructed for each system. Depending of the preconditioner this can range from almost trivial to as difficult as inverting the system itself and this is often directly correlated to how much the preconditioner reduce the solver iteration count by. This is where the idea behind this project comes from. If you could use a Machine Learning type algorithm to learn a preconditioner, to capture some potential underlying structure of a set of similar linear systems, such that the preconditioner on average works well on all of the systems. You would only need to learn a preconditioner once for each set of similar systems. This is what we will try to accomplish in this thesis.

2 Iterative methods for linear systems

Before we attempt to learn a preconditioner, we first need background theory. Here we will go through Conjugate Gradient, its convergence rate, and variants of Conjugate Gradient. And lastly a comparison of the different variants to determine which method to use as the solver for the learning

2.1 Conjugate Gradient

The Conjugate Gradients method, often shortened to CG, is an iterative method for solving linear systems of the form $Ax = b$, where A is an $n \times n$ invertible positive definite symmetric matrix, x and b being vectors of size n , without explicit calculating A^{-1} . To explain how CG work we need to start with another iterative solver Richardson iterations, which is a uses a simple recursive equation to find an approximation for the value of x

$$x_{i+1} = x_i + r_i, \quad (1)$$

with r_i being the residual defined as $r_i = b - Ax_i$ and a initial x_0 . This method work on the condition that $\|I - A\| < 1$, [8], which is for a matrix $A \in \mathbb{R}^{n \times n}$ the matrix norm defined as

$$\|A\| = \sup_{y \in \mathbb{R}^n, y \neq 0} \frac{\|Ay\|}{\|y\|}.$$

With a given x_0 we can see that the recursive equation, (1), is the sum of all previous residuals and x_0 , [8]

$$\begin{aligned} x_{i+1} &= x_{i-1} + r_{i-1} + r_i = x_0 + r_0 + r_1 + \dots + r_i \\ &= x_0 + \sum_{j=0}^i (I - A)^j r_0 \end{aligned}$$

Therefore in the subspace span by $x_0 + \{r_0, Ar_0, \dots, A^i r_0\}$, where the right part is called the i th dimensional Krylov subspace generated by A and r_0 , denoted as $\mathcal{K}^i(A, r_0)$. Which means that Richardson iterations approximation comes from an initial approximation x_0 plus a Krylov subspaces of increasing dimension. We can now almost explain CG as it a Krylov subspace method with Ritz-Galerkin condition for the residuals, which is that x_i is constructed such that the corresponding residual $b - Ax_i$ is orthogonal to the current Krylov subspace, [8].

$$b - Ax_i \perp \mathcal{K}^i(A; r_0). \quad (2)$$

This can be done by creating an orthonormal basis for the Krylov subspace with r_0 as the initial vector, this is typically done with Arnoldi algorithm, [7]. This algorithm is a modified version of the standard Gram-Schmidt procedure for creating orthonormal basis for a vector space. The procedure can be seen in Algorithm 1.

Algorithm 1 Arnoldi algorithm [8].

```

 $v_1 = \frac{r_0}{\|r_0\|_2}$ 
loop  $j = 1, \dots, m - 1$ 
     $t = Av_j$ 
    loop  $i = 1, \dots, j$ 
         $h_{i,j} = v_i^T t$ 
         $t = t - h_{i,j} v_i$ 
    end loop
     $h_{j+1,j} = \|t\|_2$ 
     $v_{j+1} = \frac{t}{h_{j+1,1}}$ 
end loop
```

With Arnoldi's algorithm we get a relation between the linear systems matrix A , a matrix V_m created from the basis vectors v_1 to v_m as columns and an $m \times m - 1$ upper Hessenberg matrix $H_{m,m-1}$, which is a matrix with all zero entries below the first subdiagonal, with $h_{i,j}$ as entries, $H_{m-1,m-1}$ denotes $H_{m,m-1}$ without the last row,

$$AV_{m-1} = V_m H_{m,m-1} \quad (3)$$

$$= V_{m-1} H_{m-1,m-1} + h_{m,m-1} v_m e_m^T \quad (4)$$

$$V_{m-1}^T AV_{m-1} = H_{m-1,m-1} \quad (5)$$

Note that if A is symmetric then so is $H_{m-1,m-1}$

$$\begin{aligned} H_{m-1,m-1}^T &= (V_{m-1}^T AV_{m-1})^T \\ &= V_{m-1}^T A^T (V_{m-1}^T)^T \\ &= V_{m-1}^T AV_{m-1} \\ &= H_{m-1,m-1} \end{aligned}$$

and because $H_{m-1,m-1}$ is a Hessenberg matrix, $H_{m-1,m-1}$ is a tridiagonal matrix for symmetric A . This is important for the recursive relations of the Conjugate Gradients.

With this orthonormal basis a solution to the linear system would consist of find linear combination, y_m to satisfy $x_m = x_0 + V_m y_m$. With the Ritz-Galerkin condition and (5) this can be done with

$$\begin{aligned} 0 &= V_m^T (b - Ax_m) \\ &= V_m^T b - V_m^T A(x_0 + V_m y_m) \\ &= V_m^T (b - Ax_0) - V_m^T AV_m y_m \\ &= V_m^T r_0 - H_{m,m} y_m \end{aligned}$$

then by $r_0 = \|r_0\|_2 v_1$ and that V_m consists of an orthonormal basis v_i , $V_m^T r_0 = \|r_0\|_2 e_1$, with e_1 being the first canonical unit vector in $\mathbb{R}^m$, we get

$$\begin{aligned} H_{m,m} y_m &= \|r_0\|_2 e_1 \\ y_m &= \|r_0\|_2 H_{m,m}^{-1} e_1. \end{aligned}$$

With this it can also be shown that the residuals r_m is proportional to the basis vector v_{m+1} , [7]. This means that the residuals make up a orthogonal basis for $\mathcal{K}^m(A; r_0)$ and therefore we can rewrite of the previous equations with the residuals as the basis vectors, R_m , and tridiagonal matrix $T_{m+1,m}$ defined as $H_{m+1,m}$

$$AR_m = R_{m+1} T_{m+1,m} \quad (6)$$

$$R_m^T AR_m = R_m^T R_{m+1} T_{m+1,m} \quad (7)$$

and

$$x_m = x_0 + R_m y_m \quad (8)$$

$$y_m = T_{m,m}^{-1} e_1 \quad (9)$$

With all this we can finally derive the recursion relations for CG. The first thing to see is that $R_m^T R_m$ is a diagonal matrix with the squared norms of the residuals on the diagonal, then (7) shows that $T_{m,m}$ can be transformed into by rowscaling into symmetric positive definite (SPD) matrix, because A is SPD. This means that $T_{m,m}$ can be LU decomposed without pivoting [8].

$$T_{m,m} = L_m U_m,$$

with L_m being lower bidiagonal and U_m being upper bidiagonal with unit diagonal. Then we can then rewrite (8)

$$x_m = x_0 + (R_m U_m^{-1})(L_m^{-1} e_1) \quad (10)$$

By the fact that bidiagonal matrices are nonsingular if all main diagonal elements are nonzero and a closed form exists for the inverse [5], the vector $\bar{\alpha} := L_m^{-1} e_1$ can be calculated recursively, with $\delta_0, \delta_1, \dots, \delta_m$ on the main diagonal and $\phi_0, \phi_1, \dots, \phi_{m-2}$ on the subdiagonal of L_m , as

$$\begin{aligned} \alpha_0 &= \frac{1}{\delta_0} \\ \alpha_i &= -\frac{\phi_{i-1} \alpha_{i-1}}{\delta_i}. \end{aligned}$$

Then for $P_m := R_m U_m^{-1} \implies R_m = P_m U_m$ with P_m consists of the vectors $p_0, \dots, p_{m-1}$ as columns and U_m being a upper bidiagonal matrix with unit diagonal and $\beta_0, \dots, \beta_{m-2}$ on the superdiagonal, implies that the residuals in the columns of R_m can be calculated as

$$r_i = \alpha_{i-1} A p_{i-1} + p_i \quad (11)$$

therefore p_i is defined as

$$p_i = r_i - \beta_{i-1} p_{i-1}. \quad (12)$$

With these recurve relations (10) can be written as, [8],

$$x_m = x_{m-1} + \alpha_{m-1} p_{m-1}. \quad (13)$$

With an initial x_0 and $p_0 := r_0$ the equations (13), (11), and (12) defines the recurrence relations for CG. Only missing the values of α_i and β_i which can be shown to be equal to, [8]

$$\alpha_{i-1} = \frac{\rho_i}{p_{i-1}^T A p_{i-1}}, \quad (14)$$

and

$$\beta_{i-1} = \frac{\rho_i}{\rho_{i-1}}, \quad (15)$$

with

$$\rho_i = r_i^T r_i. \quad (16)$$

With these values of α_{i-1} and β_{i-1} there is the possibility of zero division. For the latter there is the inner product of r_{i-1} with it self, $r_{i-1}^T r_{i-1}$. Because it is an inner product it can only equal zero if r_{i-1} is the zero vector. This will not happen in implementation, as the standard stopping condition for CG is when the norm of the newest residual is smaller than a given tolerance. As for α_{i-1} because A is SPD, $(p_{i-1}, p_{i-1})_A = p_{i-1}^T A p_{i-1}$ defines a proper inner product. Therefore as before will not equal zero in implementation.

2.1.1 Convergece

In exact arithmetic, meaning that there is no rounding, CG will converge at the latest n iterations for a linear system of size n . This is because the residuals created in each iteration form an orthogonal basis for the Krylov subspace of increasing dimension. therefore a maximum of n residuals can be made before the Krylov subspace coincides with $\mathbb{R}^n$. In implementation this only occurs for well conditioned matrices,

meaning a matrix with low condition number. Where the condition number of an invertible matrix can be defined as, [8],

$$\kappa(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\lambda_{max}}{\lambda_{min}}. \quad (17)$$

To show a practical convergence rate of CG will find a bound for the A -norm of the error at i 'th iteration, $\|x_i - x\|_A$, where x_i is the i 'th approximation and x is the exact solution.

For a vector $u \in x_0 + \mathcal{K}^i(A; r_0)$ and a polynomial Q_i of degree i and $Q_i(0) = 1$ then

$$x - u = Q_i(A)(x - x_0). \quad (18)$$

This is proven by $u \in x_0 + \mathcal{K}^i(A; r_0) \implies u = x_0 + \hat{Q}_{i-1}(A)r_0$, $r_0 = b - Ax_0$ and $Ax = b$, implies that

$$\begin{aligned} x - u &= x - (x_0 + \hat{Q}_{i-1}(A)r_0) \\ &= x - (x_0 + \hat{Q}_{i-1}(A)(Ax - Ax_0)). \end{aligned}$$

Then define $\tilde{Q}_i(A) = \hat{Q}_{i-1}(A)A$ with $\tilde{Q}_i(0) = 0$

$$\begin{aligned} &= (x - x_0) - \tilde{Q}_i(A)(x - x_0) \\ &= (I - \tilde{Q}_i(A))(x - x_0) \end{aligned}$$

define $Q_i(A) = I - \tilde{Q}_i(A)$ with $Q_i(0) = 1$

$$= Q_i(A)(x - x_0).$$

This means that by (18) we can write the i 'th iterations error as a initial error times a polynomial of degree i ,

$$x_i - x = P_i(A)(x_0 - x). \quad (19)$$

Next denote the eigenvalues and orthonormalized eigenvectors of A as λ_j and z_j , which means we can write r_0 as a linear combination of the eigenvectors $r_0 = \sum_j \gamma_j z_j$ with scalars γ_j and because r_i can be written as polynomial of A of degree i with $P(0) = 1$ times the initial residual r_0 , [8], therefore r_i can be written as

$$r_i = P_i(A)r_0 = \sum_j \gamma_j P_i(\lambda_j) z_j. \quad (20)$$

We can now find a bound for the squared norm of the i 'th iterations error

$$\begin{aligned} \|x_i - x\|_A^2 &= \|P_i(A)(x_0 - x)\|_A^2 \\ &= (x_0 - x)^T P_i(A) A P_i(A) (x_0 - x) \\ &= (x_0 - x)^T P_i^T(A) P_i(A) (Ax_0 - Ax) \end{aligned}$$

When by $r_i = b - Ax_i \implies Ax_i = b - r_i$ and $Ax = b$

$$= (x_0 - x)^T P_i^T(A) P_i(A) (-r_0)$$

when by (20)

$$\begin{aligned} &= (x - x_0)^T P_i^T(A) \sum_j \gamma_j P_i(\lambda_j) z_j \\ &= (x - x_0)^T A^T A^{-T} P_i^T(A) \sum_j \gamma_j P_i(\lambda_j) z_j \\ &= (A(x - x_0))^T A^{-T} P_i^T(A) \sum_j \gamma_j P_i(\lambda_j) z_j \\ &= (r_0)^T P_i^T(A) A^{-T} \sum_j \gamma_j P_i(\lambda_j) z_j \end{aligned}$$

again with the transpose of (20)

$$= \sum_l (\gamma_l P_i(\lambda_l) z_l) A^{-T} \sum_j \gamma_j P_i(\lambda_j) z_j$$

because $Az_l = \lambda_l z_l \implies A^{-1}z_l = \frac{1}{\lambda_l} z_l$

$$= \sum_l \left(\frac{\gamma_l}{\lambda_l} P_i(\lambda_l) z_l \right) \sum_j \gamma_j P_i(\lambda_j) z_j$$

Now because the eigenvectors z_j form an orthonormal basis we finally get

$$= \sum_j \frac{\gamma_j^2}{\lambda_j} P_i(\lambda_j)$$

The sum can now be bounded by the maximum of $|P_i(\lambda_j)|$ and instead of the optimal polynomial P_i using an arbitrary polynomial Q_i of degree i with $Q_i(0) = 1$.

$$\sum_j \frac{\gamma_j^2}{\lambda_j} P_i(\lambda_j) \leq \max_j |Q_i(\lambda_j)| \sum_j \frac{\gamma_j^2}{\lambda_j}$$

With a similar derivation as the previous, it can be shown that $\sum_j \frac{\gamma_j^2}{\lambda_j} = \|x_0 - x\|_A^2$.

$$\|x_i - x\|_A^2 \leq \max_j |Q_i(\lambda_j)| \|x_0 - x\|_A^2$$

By replacing the arbitrary polynomial Q_i with the Chebyshev polynomial C_i transformed and scaled such that when evaluated at zero it equals 1,

$$Q_i(\lambda) = \frac{C_i\left(\frac{2\lambda - (\lambda_{min} + \lambda_{max})}{\lambda_{max} - \lambda_{min}}\right)}{C_i\left(-\frac{\lambda_{min} + \lambda_{max}}{\lambda_{max} - \lambda_{min}}\right)}$$

where λ_{min} and λ_{max} are the minimum and maximum eigenvalues. We get the bound, [8],

$$\|x_i - x\|_A^2 \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^i \|x_0 - x\|_A^2 \quad (21)$$

where $\kappa(A)$ is called the condition number of A and is defined as

$$\kappa(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\lambda_{max}}{\lambda_{min}}$$

From this we can conclude that the convergence rate of CG for matrices with higher condition number has a slower convergence than matrix with a low condition number.

2.1.2 Preconditioning

Because the convergence rate of CG depends on the condition number, if the linear system is transformed such that the new systems condition number is smaller would result in faster convergence. This is called preconditioning the linear system. More specific left preconditioning by a matrix M is to transform the system $Ax = b$ as such

$$M^{-1}Ax = M^{-1}b,$$

with the goal that

$$\kappa(M^{-1}A) \ll \kappa(A).$$

Because of the condition that the matrix needs to be SPD, the transformed system needs to be SPD and because $M^{-1}A$ is only symmetric if both are symmetric and they commute. This is because a matrix B is symmetric if and only if

$$(Bx, y) = (x, By), \quad \forall x, y \in \mathbb{R}^n.$$

To ensure the preconditioned systems is SPD, we can change the inner product we use in the recursion relations to

$$[x, y] := (x, My). \quad (22)$$

This inner product is a proper inner product if M is positive definite. With this change in inner product, $M^{-1}A$ is symmetric

$$\begin{aligned} [M^{-1}Ax, y] &= (M^{-1}Ax, My) \\ &= (Ax, M^{-1}My) \\ &= (x, Ay) \\ &= (x, MM^{-1}Ay) \\ &= [x, M^{-1}Ay]. \end{aligned}$$

The new recurrence relations for CG therefore be written as with $r_0 = M^{-1}(b - Ax_0)$ and $p_0 = w_0$

$$\begin{aligned} x_i &= x_{i-1} + \alpha_i p_i \\ r_i &= r_{i-1} - \alpha_i A p_i \\ p_i &= w_{i-1} + \beta_{i-1} p_{i-1} \\ w_i &= M^{-1} r_i \end{aligned}$$

with scalars

$$\begin{aligned} \alpha_i &= \frac{\rho_{i-1}}{p_i^T A p_i} \\ \beta_i &= \frac{\rho_i}{\rho_{i-1}} \\ \rho_i &= r_i^T w_i. \end{aligned}$$

The new relations have an extra matrix vector product compared to the unpreconditioned version. Because of this it is important that the decrease in condition number is large enough such that the extra computational cost needed does not make actual run time of the algorithm longer.

An example of a preconditioner is the (point) Jacobi preconditioner, which is constructed from the diagonal of the original system, [2], meaning for matrix $A = (a_{i,j})$ the preconditioner $M = (m_{i,j})$ is defined as

$$m_{i,j} = \begin{cases} a_{i,j} & \text{if } i = j \\ 0 & \text{otherwise} \end{cases} \quad (23)$$

This preconditioner has the advantage that it is sparse. It can therefore have fast matrix vector product if constructed as a compressed sparse row matrix (CSR matrix) which has a matrix vector product computational cost depended on the number of nonzero entries instead of the size of the matrix for the standard implementation [1].

2.2 CG Variants

Because of the conditions for Conjugate Gradient that the system matrix and the preconditioner must be symmetric positive definite, we will look at iterative methods derived from CG where these conditions are not needed for the system and preconditioner. This does come with disadvantages for the different variants, for example irregular convergence, increased computational cost and breakdown conditions.

2.2.1 Normal Equation

One way to solve the linear system, $Ax = b$, where the matrix, A , is not symmetric, is to solve an equivalent system, known as the system of the normal equations

$$A^T A x = A^T b \quad (24)$$

this systems matrix, $A^T A$, is positive definite

$$x^T A^T A x = \|Ax\|^2 \geq 0, \forall x \in \mathbb{R} \setminus \{0\}, \quad (25)$$

and can only take a value of zero if A is a matrix of all zeros and therefore also singular. The new system matrix is also symmetric

$$(A^T A)^T = A^T (A^T)^T = A^T A. \quad (26)$$

Which means the transformed system can be solved with CG, but it does not come for free. It either needs an extra matrix vector product needed in each iteration of the algorithm, with the transpose of the matrix, which the difficulty of is depend on the implementation of the matrix. Or a matrix matrix product before all iterations which is a costly operator even when only done once.

A bigger problem is the condition number of the new matrix is the square of the original matrix, [7],

$$\kappa(A^T A) = \kappa(A)^2,$$

meaning that convergence will be slower, and with the extra matrix vector product the run time of the algorithm will increase. One could then precondition the system, as described in section 2.1.2, to lower the condition number. This still has the condition that the preconditioner needs to be SPD for the CG to work on the preconditioned system.

2.2.2 Bi-Conjugate Gradient

Another way to solve a system with unsymmetric A , is with the Petrov-Galerkin approach, [8], which is to find x_m such that the residual r_m is orthogonal to another m -dimensional subspace. The choice for this method is subspace is the Krylov subspace generated by A^T and r_0 , $\mathcal{K}^m(A^T; r_0)$, which will lead to the iterative solver known as Bi-Conjugate Gradient (Bi-CG). An elegant way to derive the recurrence relations for this method is to use CG on this specific system, [8],

$$\begin{pmatrix} 0 & A \\ A^T & 0 \end{pmatrix} \begin{pmatrix} \hat{x} \\ x \end{pmatrix} = \begin{pmatrix} b \\ \hat{b} \end{pmatrix}. \quad (27)$$

with the preconditioner

$$\hat{M} = \begin{pmatrix} 0 & M \\ M^T & 0 \end{pmatrix}, \quad (28)$$

written as $\hat{M}^{-1} B \tilde{x} = \hat{M}^{-1} \tilde{b}$

With this construction CG can be used to solve the system since $\hat{M}^{-1}B$ is symmetric for the inner product $[x, y] := (x, My)$, which is not a proper inner product since $\hat{M}^{-1}B$ is not positive definite, [8]. Because it is not a inner product this shows a problem with Bi-CG, as this can be evaluated to zero when a proper inner product wouldn't, this is known as breakdown of the method.

Which gets us the relations,

$$\begin{aligned} p_i &= M^{-1}r_{i-1} + \beta_{i-1}p_{i-1} \\ \tilde{p}_i &= M^{-T}\tilde{r}_{i-1} + \beta_{i-1}\tilde{p}_{i-1} \\ x_i &= x_{i-1} + \alpha_i p_i \\ r_i &= r_{i-1} - \alpha_i A p_i \\ \tilde{r}_i &= \tilde{r}_{i-1} - \alpha_i A^T \tilde{p}_i \end{aligned}$$

with scalars

$$\begin{aligned} \alpha_i &= \frac{\rho_{i-1}}{\tilde{p}_i^T A p_i} \\ \beta_i &= \frac{\rho_{i-1}}{r h o_{i-2}} \\ \rho_{i-1} &= (M^{-T}\tilde{r}_{i-1})^T M^{-1}r_{i-1} \end{aligned}$$

with initial residual $r_0 = b - Ax_0$ and $\tilde{r}_0$ such that $(r_0, \tilde{r}_0) \neq 0$, often chosen to be $\tilde{r}_0 = r_0$. With the bi-orthogonality and bi-conjugate, between the two sets of vectors, [7],

$$(r_j, \tilde{r}_i) = 0, \text{ for } j \neq i \quad (29)$$

$$(A p_j, \tilde{p}_i) = 0, \text{ for } j \neq i, \quad (30)$$

ensures that Bi-CG would converge in exact arithmetic. As mentioned breakdown can occur with Bi-CG, other disadvantages with this method is the extra matrix vector products and as with using the system of normal equations is the need to be able to calculate the transpose of the original systems matrix, as said the difficulty depends on the implementation of the matrix. A more serious problem with Bi-CG and the methods derived from it, is that in implementation the methods suffer from irregular convergence. This means that norm of the current residual may be larger than the previous residual, even if the algorithm does converge.

2.2.3 Conjugate Gradient Squared

Conjugate Gradient Squared (CGS) is a variant of Bi-CG, that was made with main purpose to not have calculate the transpose of the system matrix A , [7]. We will not go through derivation, but will give the idea behind it. The derivations is based on the fact the both sets of residuals in Bi-CG is calculated implicitly as the product of a polynomial and the starting residual, [8],

$$\begin{aligned} r_i &= P_i(A)r_0 \\ \tilde{r}_j &= P_j(A^T)\tilde{r}_0, \end{aligned}$$

and by the bi-orthogonality of the residuals in Bi-CG, (29),

$$\begin{aligned} 0 &= (r_j, \tilde{r}_i) \\ &= (P_i(A)r_0, P_j(A)\tilde{r}_0) \\ &= (P_j(A)P_i(A)r_0, \tilde{r}_0). \end{aligned}$$

Then changing to using the residual defined as $\hat{r}_i = P_i^2(A)r_0$. With this the CGS would not have calculate the transpose of the matrix and the calculating the set of secondary residuals, $\tilde{r}_i$, is not needed, [8]. Because we will not use this method we will not show the recursion relations. This would have been shown that the computational cost of CGS is similar to Bi-CG, but generally faster convergence. It still suffers from breakdowns and irregular convergence similar to Bi-CG.

2.2.4 Bi-Conjugate Gradient Stabilized

The last variant of CG we will look at is Bi-Conjugate Gradient Stabilized (Bi-CGStab). The derivation uses the same idea as CGS with changing the residual from Bi-CG with the help of bi-orthogonality between the two sets of residuals. Then instead of combining the two polynomials into one, $\hat{r}_i = P_i^2(A)r_0$, changing the second polynomial to a different polynomial such that it minimizes r_i and the recursion relations with the polynomial is simple to derive, [7]. For Bi-CGStab the polynomial is of the form

$$Q_i(X) = (I - \omega_i X)Q_{i-1}(X) \quad (31)$$

the resulting residual is therefore of the form

$$\hat{r}_i = Q_i(A)P_i(A)r_0. \quad (32)$$

To derive the recursion relation for this new residual we will use the polynomial form of r_i and p_i from Bi-CG, with polynomials P_i and T_{i-1} of degree i and $i-1$ respectively

$$r_i = P_i(A)r_0 \quad P_i(A)r_0 = (P_{i-1}(A) - \alpha_i A T_{i-1}(A))r_0 \quad (33)$$

$$p_i = T_{i-1}(A)r_0 \quad T_{i-1}(A)r_0 = (P_{i-1}(A) + \beta_{i-1} T_{i-2}(A))r_0 \quad (34)$$

With (33) and the definition of Q_i (31), the recursion relation for the residual can be derived with substitution in (32),

$$Q_i(A)P_i(A)r_0 = (I - \omega_i A)(Q_{i-1}(A)P_{i-1}(A) - \alpha_i A Q_{i-1}(A)T_{i-1}(A))r_0$$

with the need to derive what the product of $Q_i(A)T_i(A)r_0$. Which can be derived with substitution with (34) to get

$$Q_i(A)T_i(A)r_0 = (Q_i(A)P_i(A) + \beta_i(I - \omega_i A)Q_{i-1}(A)T_{i-1}(A))r_0.$$

With $\hat{p}_i = Q_{i-1}(A)T_{i-1}(A)r_0$, the vector form of $\hat{p}_i$ and $\hat{r}_i$ is equal to

$$\hat{p}_i = r_{i-1} + \beta_i(I - \omega_{i-1}A)\hat{p}_{i-1} \quad (35)$$

$$\hat{r}_i = (I - \omega_i A)(\hat{r}_{i-1} - \alpha_i A \hat{p}_i) \quad (36)$$

For the equation for x_{i-1} we first need to define $s_i = \hat{r}_i - \alpha_i A \hat{p}_i$ then rewrite (36) as such

$$\hat{r}_i = \hat{r}_{i-1} - \alpha_{i-1} A \hat{p}_{i-1} - \omega_{i-1} A s_{i-1}. \quad (37)$$

Which gets us the update relation for x_i

$$x_i = x_{i-1} + \alpha_{i-1} \hat{p}_{i-1} + \omega s_{i-1}. \quad (38)$$

Lastly the scalars α_i , β_i , ω_i and ρ_i can be shown to be equal to

$$\alpha_i = \frac{\rho_{i-1}}{\tilde{r}^T A \hat{p}_i} \quad (39)$$

$$\beta_i = \frac{\rho_{i-1} \alpha_{i-1}}{\rho_{i-2} \omega_{i-1}} \quad (40)$$

$$\omega_i = \frac{(As)^T s}{(As)^T As} \quad (41)$$

$$\rho_i = \tilde{r}^T \hat{r}_{i-1} \quad (42)$$

From the orthogonality from the start of the derivation, $P_i(A)r_0, Q_j(A^T)\hat{r}_0 = 0$, says that Bi-CGStab will terminate in exact arithmetic. The whole derivation is based on [8].

2.2.5 Comparison

The computational cost of each iteration for the different method mentioned can be see in table 1. From this table we can see that CG is the computational cheapest of the five methods, but with the condition that the system matrix and the preconditioner are both SPD. For the variants of CG we first have CG on the system of normal equations can have 2 extra matrix vector product depending on implementation, but has the problem of squaring the condition number making the converges slower. Bi-CG has the similar computational cost as CG Normal with the without squaring the condition number. But can suffer from breakdowns and has irregular convergence of the residuals. Next CGS does not need the transpose and in practice has faster converges compared to Bi-CG, but still suffers from irregular convergence and breakdowns. With the highest computational cost of the five methods is Bi-CGStab, with similar convergence rate to CGS but less irregular convergence compared to it. For the computational cost the most costly is the matrix vector product for which all beside standard CG uses 2 for the system matrix and 2 for the preconditioner, therefore the differences in cost between the variants are similar, [2]. Therefore for these reasons and that it does not need the preconditioner to be SPD, we will use Bi-CGStab as the method in our exploration of trying to learn a preconditioner.

Method	Inner products	$\alpha x + y$	Matrix vector product	Precond matrix vector products
CG	2	3	1	1
CG Normal	2	3	1 or (1/1)	1 or (1/1)
Bi-CG	2	5	1/1	1/1
CGS	2	6	2	2
Bi-CGStab	4	6	2	2

Table 1: The computational cost for an iteration for each of the five methods [2]. With columns corresponding to in order the name of the method; number of inner products; number of vector scaling and addition; number of matrix vector products; number of precondition matrix vector products. a/b indicate matrix vector products with the matrix itself and its transpose respectably. a (b/c) depends on the implementation.

3 Exploration of learning preconditioners

Now that we have the necessary theory about the iterative solver and their convergence. The two first thing that need to be done in this exploration of learning a preconditioner, is to determine the structure of our preconditioner and the algorithm that will learn the preconditioner

3.1 Structure of the preconditioner

The two main structure properties that the preconditioner must have is, first that is easy to apply to a vector. As mentioned in section 2.1.2 the extra matrix vector products that is need to be done in each of the solvers iterations, regardless of the specific solver used, but is more prevalent for the more complicated methods. The simplest way to ensure a lower computational cost of a matrix vector product is to use preconditioner which is sparse. It can therefore be implemented as a CSR Matrix, which has the computational cost of the matrix vector product proportional to the number of nonzero values in the matrix [1]. Another benefit from limiting to strictly sparse preconditioning matrices is the storage requirement is decreased significantly.

The second is the degrees of freedom must be kept relatively low, as what we are trying to achieve is to learn a preconditioner that lowers the solver iteration count. The key point is to learn, as the number of degrees of freedom determines how may different preconditioning matrices that can be found. This is a balance between of how fast it can it can explore the space of matrices, which in theory equates to faster learning. With fewer degrees of freedom. Against slower learning but with the possibility of finding a better preconditioner. With a higher degree of freedom.

The structure we have chosen to achieve these properties is to use an identity diagonal and partition the 1st superdiagonal into m as equal sized parts as possible. Because the matrix shift each elements of a vector by the next elements times a coefficient, we will call this preconditioning a partitioned shift preconditioner. The sparsity of this type of matrix is clear, as it has 2 nonzero elements per row beside the last row with 1 nonzero element. With this construction it is easy to control the degrees of freedom for testing different sizes, as it is equal to the number of partitions m .

$$\begin{pmatrix} 1 & x_1 & 0 & & & \\ 0 & 1 & x_1 & 0 & & \\ & 0 & 1 & x_2 & 0 & \\ & & & \ddots & \ddots & \\ & & & 0 & 1 & x_{m-1} & 0 \\ & & & & 0 & 1 & x_m \\ & & & & & 0 & 1 \end{pmatrix} \quad (43)$$

To test if this preconditioner has any chance of working, we chose four different matrices from Matrix Market [6], which is repository of sparse matrices. We will then precondition these four matrices with 500 different preconditioning matrices, to see if any of the preconditioned systems solver iteration count is less than the system without a preconditioner applied to it. For all four system the vector b will be an all ones appropriate size. Each of the four matrices will have slightly different preconditioners, with different number of partitions and different distribution for generating the coefficients for the partitions. The relevant information for both the matrices and their preconditioner can be seen in table 2. The four matrices was chosen such that we no matrix with the same size and condition number. To get consistent results any randomness is seeded and all code is run the uCloud servers, as code ran on different computing configurations can yield different results. This is also to take advantage of parallel computing, as each of the different preconditioned system is independent, meaning each system can be solved with Bi-CGStab independently.

To determine the distribution of partition coefficients, we ran the code on a small subset of the preconditioned systems. This is done because the solver we are using, Bi-CGStab, as described in section 2.2.4 can suffer from breakdowns and is more prevalent when the system becomes ill-conditioned proportional

to the size of the system. The starting distribution of this preliminary test was the normal distribution with mean 0 and variance 1. Here very few systems converged and those that did the coefficients of the partitions were generally closer to zero, in order of $|10^{-2}|$ than to ± 1 . From this we can conclude that a smaller standard deviation such that the generated values have similar orders of magnitude as those that converged would most likely to have fewer systems that will suffer from breakdowns and converges successfully. For the number of partitions these preliminary test did not give any conclusive results as to the number of partition that gave a better preconditioner. The choice of preconditioner parameters can be seen in table 2

Name	Dimension	Condition number (est)	Number of partitions	Distribution
eris1176	1176	Not reported	15	$N(0, 0.1^2)$
fidap004	1601	$5.17 \cdot 10^3$	15	$N(0, 0.05^2)$
orsirr_1	1030	$1 \cdot 10^2$	10	$N(0, 0.05^2)$
sherman5	3312	$3.9 \cdot 10^5$	10	$N(0, 0.1^2)$

Table 2: Table of the Matrix Market [6] matrices with names; the size of the matrix; the reported condition number; the number of partitions in the superdiagonal of the preconditioner; the distribution that generated the coefficients.

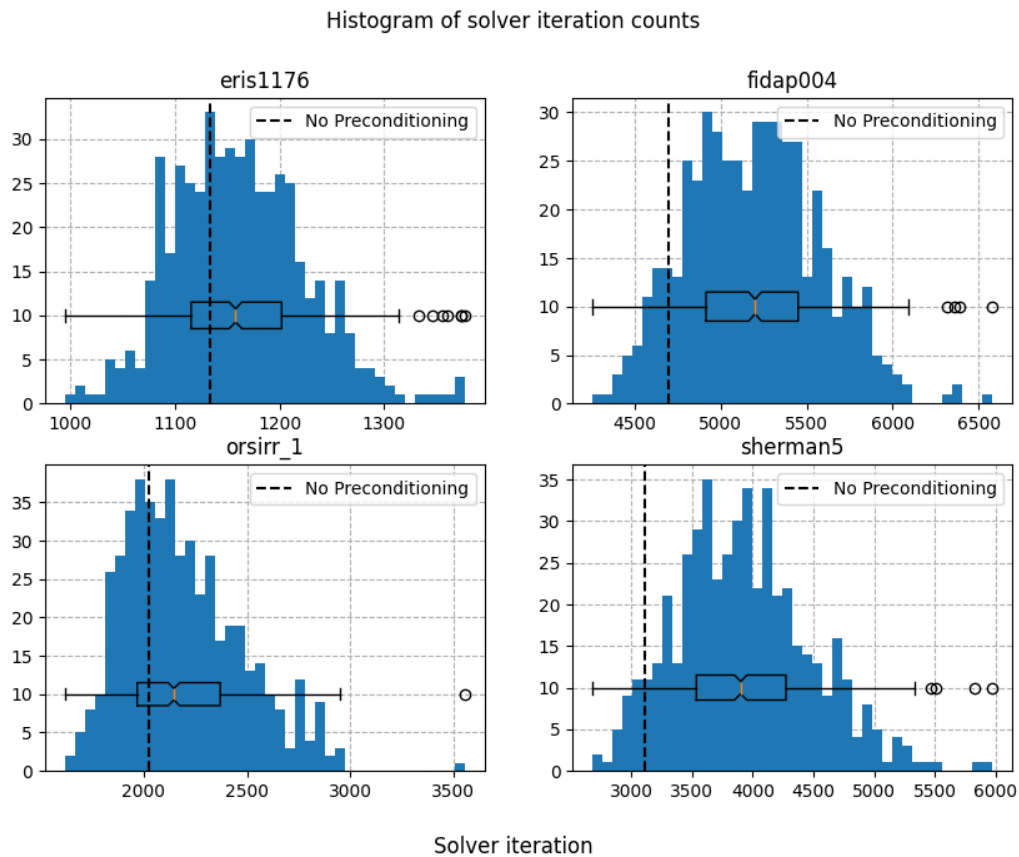


Figure 1: Histogram and boxplot of the solver iterations counts of 500 preconditioned systems for each of the different Matrix Market matrices with a plot for each, with a dashed line corresponding to the unpreconditioned system solver iteration count.

Matrix	Without preconditioning	Mean	Median	std	Number with lower iteration count	Breakdowns
eris1176	1134	1159.39	1157	81.85	175	1
fidap004	4695	5197.11	5200.5	384.88	47	0
orsirr_1	2025	2190.75	2146	287.80	163	0
sherman5	3115	3904.15	3898.5	606.91	31	5

Table 3: Solver iteration count statics of the four matrices preconditioned systems. With each column in order corresponding to the name of the matrix; the solver iteration count for the unpreconditioned system; the mean, median and standard deviation preconditioned solver iteration counts; the number of preconditioned matrices which have a had a solver iteration count lower than the unpreconditioned system; number of preconditioned system that suffered from a breakdown.

From both the histograms in figure 1 and the second to last column of table 3, showing how many of the preconditioned systems had a fewer number of solver iterations, we can see that partitioned shift preconditioner can has the possibility of working as preconditioner given the right coefficients. And that the condition number of the original system plays a part in how well the preconditioner works, as the two matrices with the highest condition number has the fewest number preconditioners that improved the solver iteration count, this a bit unexpected as system with a higher base solve iteration count has larger potential for a improvements. It could also mean that the coefficient generations might have a bad distribution to generate a good preconditioner for these two specific systems. This is where the learning of the preconditioner comes in, because then the initial coefficient should not matter, but only if the initial preconditioner would cause a breakdown of the algorithm to occur.

3.2 Initial learning algorithm

We will now construct a learning algorithm that can find a set of coefficients for the partitions of the preconditioner such that the preconditioned system have a decreased solver iteration count. Then we will test the algorithm on the four matrices from the previous section. The reason we construct a leaning algorithm and not using an already established Machine Learning algorithm is the need to define a loss function. The standard loss functions expect a continues input, if then we use the solver iteration count as the input to those functions it might run into unexpected behavior. And because the condition number is only a bound for the error it is not the ideal measure of improvement.

The approach we take to learning the coefficients of the preconditioner takes inspiration from stochastic gradient decent. Starting with randomly generated initial coefficients for the partitions with a predetermined number of partitions. Then choose one of the coefficient at random to be change by value generated from some distribution. With this new changed preconditioner the new system is solved to get a solver iteration count to use as a measure, to determine of the change made to the coefficient had a positive or negative effect on the solver iteration. If it decreases the change is kept, otherwise if it increases the iteration count it is rejected. Regardless of the outcome a new coefficient is chosen to be changed. This procedure is then repeated for a number of predetermined learning steps. Pseudocode for this procedure can be seen in Algorithm 2.

The new parameters that needs to be determined for the learning algorithm is: number of learning steps and how to change the partition coefficients. To choose which coefficient that will be changed we have decided on uniform distribution for which coefficient to be changed, with the knowledge that for larger number of partitions, a larger number of learning steps is needed to explore each coefficient thoroughly. For the value of the change to the coefficient we will use a uniform distribution with mean of zero and different range for each of the four test matrices. Lastly for the number of learning steps we set this to 300 for all test matrices. The number of partitions and the distribution parameters can be seen in table 4.

Algorithm 2 Preconditioner learning algorithm**Require:** Matrix A and vector b form $Ax = b$ $M^{-1} \leftarrow$ Generate initial preconditioner $k \leftarrow$ Solver iteration count for $M^{-1}Ax = M^{-1}b$ **loop** $i = 1, 2, \dots$ $M_{new}^{-1} \leftarrow$ Changed M^{-1} $k_{new} \leftarrow$ Solver iteration count for $M_{new}^{-1}Ax = M_{new}^{-1}b$ **if** $k_{new} < k$ **then** $k \leftarrow k_{new}$ $M^{-1} \leftarrow M_{new}^{-1}$ **end if****end loop**

Matrix	Number of partitions	Coefficient distribution	Change distribution
eris1176	8	$N(0, 0.1^2)$	$Uni(-0.5, 0.5)$
fidap004	8	$N(0, 0.05^2)$	$Uni(-0.1, 0.1)$
orsirr_1	15	$N(0, 0.01^2)$	$Uni(-1, 1)$
sherman5	4	$N(0, 0.05^2)$	$Uni(-0.01, 0.01)$

Table 4: The learn parameters for the different matrices with each column corresponding to in order the name of the matrix; number of partitions in the preconditioner; the distribution for the initial partition coefficient; the distribution for the changing the values of the coefficients.

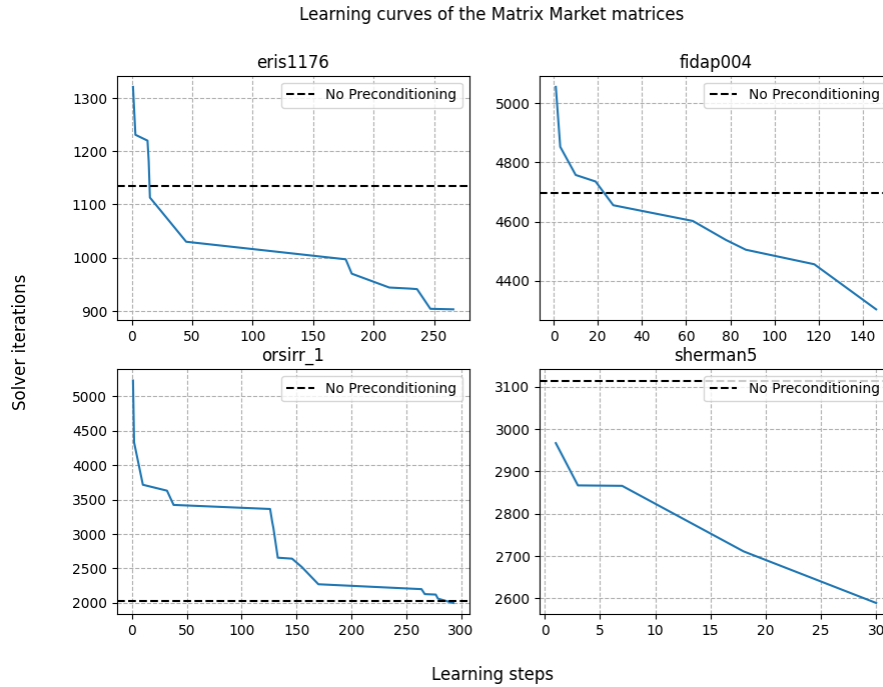


Figure 2: Learning curves of each of the four test matrices. Where the blue line shows the solver iteration count, with data point for each learning step the algorithm accepted a new preconditioner, and a dashed line indicating the unpreconditioned solver iteration count for corresponding matrix.

What immediately can be seen from the learning curves in figure 2, is that our algorithm does what we set out to do. Because it finds a better set of coefficient that form a better preconditioner and that it uses fewer solver iterations than the corresponding system with a preconditioner applied. The final solver iteration counts can be seen in table 5. Another thing, that is not immediately obvious, is that very few of the learning steps a new preconditioner got accepted, at most 17 out the 300 iterations for orsirr_1 but this may be because of the initial preconditioner making the system much worse and therefore easier to learn. It is difficult to tell if this is a real problem because of the fact the all learned preconditioner works better than with no preconditioner applied, even when the initial preconditioner makes the solver iteration count much larger. The few number of improvements for sherman5 could be contributed to the fact that the initial preconditioner already improves the system and starts at a lower solver iteration count than the unpreconditioned, so it may have learned what it can with few steps. This difficult to say from sherman5's data, but it is clear from the runs for the three other matrices that this learning method can work.

Matrix	Final	Compare v Non	Learning improvement	Number of improvements
eris1176	903	231	418	14
fidap004	4304	391	750	10
orsirr_1	1998	27	3233	17
sherman5	2589	526	378	5

Table 5: Table of data from the learning of the preconditioner with row for each for the matrices and columns corresponding to in order the name of the matrix; the solver iteration count for the final preconditioner; how many fewer solve iterations the preconditioned used compared to the unpreconditioned system; solver iteration count difference between the initial preconditioned system and the final preconditioned system; the number of times a a change to the preconditioned was accepted.

3.2.1 Better exploration of the preconditioner

As mentioned in the previous section the number of learning steps where a change was accepted. This could be because of a bad exploration of the coefficients in the partitions. The change we will make to the learning algorithm is as such: if a change made to the preconditioner did not improve the solver iteration count, we will try the same change on the same coefficient scaled by 0.5, if that worked the scaled change is accepted and continues as the original algorithm. If this scaled change also get rejected, the change will be scaled by 0.25. Then if all scaled changes does not improve the solver iteration count the algorithm continues as normal by generating a new change and a new coefficient is chosen. For pseudocode see Algorithm 3. Because the new version of the learning algorithm will test up to 3 times as many different preconditioners, therefore for the test to be comparable to the previous test we will use a base number of learning steps of 100 instead of 300. Other parameters of the algorithm will not be changed, table 4.

When comparing the result to the previous test the learned preconditioner is a small amount worse and the reason for the change to the algorithm also did not improve as the number of accepted preconditioners is also less, table 6. A positive improvement is that the learning curves are steeper meaning faster learning, figure 3. This is especially notable for the matrix fidap004 and might explained the fewer number of improvements. As now the algorithm finds the best preconditioner faster and can therefore find any more changes that would improve the preconditioner. Because the learned preconditioner are only a marginally worse and especially the faster learning, we will keep the change made to the learning algorithm.

Algorithm 3 Preconditioner learning algorithm with scaling of the change**Require:** Matrix A and vector b form $Ax = b$ $M^{-1} \leftarrow$ Generate initial preconditioner $k \leftarrow$ Solver iteration count for $M^{-1}Ax = M^{-1}b$ **loop** $i = 1, 2, \dots$ **loop** $s = \{1, 0.5, 0.25\}$ $M_{new}^{-1} \leftarrow$ Changed M^{-1} with scale s $k_{new} \leftarrow$ Solver iteration count for $M_{new}^{-1}Ax = M_{new}^{-1}b$ **if** $k_{new} < k$ **then** $k \leftarrow k_{new}$ $M^{-1} \leftarrow M_{new}^{-1}$

Break the scale for loop

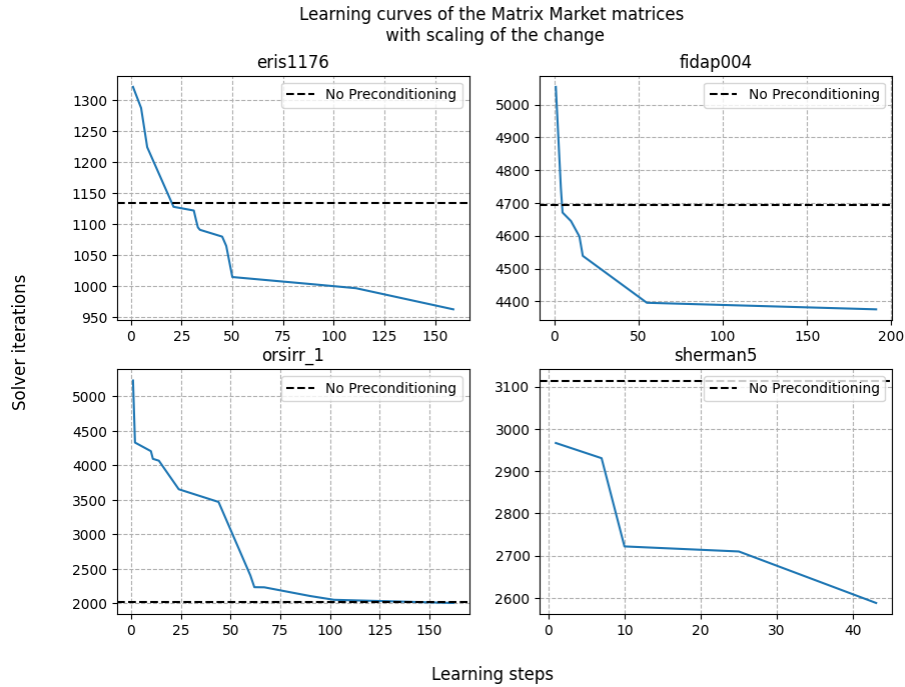
end if **end loop****end loop**

Figure 3: Learning curves with scaling the change during learning for the four different test matrices. Where the blue line shows the solver iteration count, with a data point for each learning step the algorithm accepted a new preconditioner, and a dashed line indicating the unpreconditioned solver iteration count for corresponding matrix.

Matrix	Final	Compare v Non	Learning improvement	Number of improvements
eris1176	963	171	358	14
fidap004	4376	319	678	8
orsirr_1	2007	18	3224	13
sherman5	2588	527	379	5

Table 6: Table of data from the learning of the preconditioner with scaling the change of the preconditioner with a row for each for the matrices and columns corresponds as table 5.

3.3 Discrete laplace operator

The main problem with the two previous tests and how we are using the learning algorithm, is that we only have one linear system for each run of the algorithm. A consequences of this is that the algorithm does not learn a preconditioner that could work for some general matrix, but a good preconditioner for that specific matrix, which is what this project is about. A parallel to this is that with only one matrix the algorithm is unnecessary, as we are learning to solve the system with fewer iteration by solving system itself. The only way to solve this is to get more matrices and to have a set matrices for the training part and set to evaluate the learned preconditioner, as to ensure the algorithm is not overfitting to the training data. To do this we will use the 2-dimensional discrete laplace operator, Δ , with periodic boundary conditions. As the laplace operator has a direct connection to the Dirac operator $D^2 = \Delta$.

The D -dimensional discrete laplace operator can be constructed as such [3],

$$\Delta = \sum_{d=1}^D \left(\left(\bigotimes_{k=d}^{d-1} I^{(N_k)} \right) \otimes \Delta_{h_d}^{(N_d)} \otimes \left(\bigotimes_{k=d+1}^D I^{(N_k)} \right) \right). \quad (44)$$

Where $\Delta_{h_d}^{(N_d)}$ is the $N_d \times N_d$ 1-dimensional discrete laplace operator, where we have chosen with periodic boundary conditions, for the d 'th dimension is defined as

$$\Delta_{h_d}^{(N_d)} = \frac{1}{h_d} \begin{bmatrix} -2 & 1 & 0 & \cdots & 0 & 1 \\ 1 & -2 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \\ 0 & 0 & 0 & \cdots & -2 & 1 \\ 1 & 0 & 0 & \cdots & 1 & -2 \end{bmatrix}, \quad (45)$$

with discretization step size of h_d and N_d grid points. $I^{(N_d)}$ is a $N_d \times N_d$ identity matrix. $A \otimes B$ is the Kronecker product defined between two matrices A and B of size $n \times m$ and $i \times j$ respectively as

$$A \otimes B = \begin{bmatrix} a_{11}B & \cdots & a_{1m}B \\ \vdots & \ddots & \vdots \\ a_{n1}B & \cdots & a_{nm}B \end{bmatrix},$$

and the resulting matrix is of size $n \cdot i \times m \cdot j$.

The 2-dimensional laplace can then be written as

$$\Delta_{h_1}^{(N_1)} \otimes I^{(N_2)} + I^{(N_1)} \otimes \Delta_{h_2}^{(N_2)}. \quad (46)$$

Because we want the matrices to be invertible, the matrix has to be square and the Kronecker produces a matrix of size $n \cdot i \times m \cdot j$, we will use the same discretization for both dimensions, $N_1 = N_2 = N$ and for simplicity both of the discretization step sizes will be set to one, $h_1 = h_2 = 1$. An example with $N = 6$ the 2-dimension discrete laplace operator can be seen in figure 4 as a heatmap, with zeros removed for clarity .

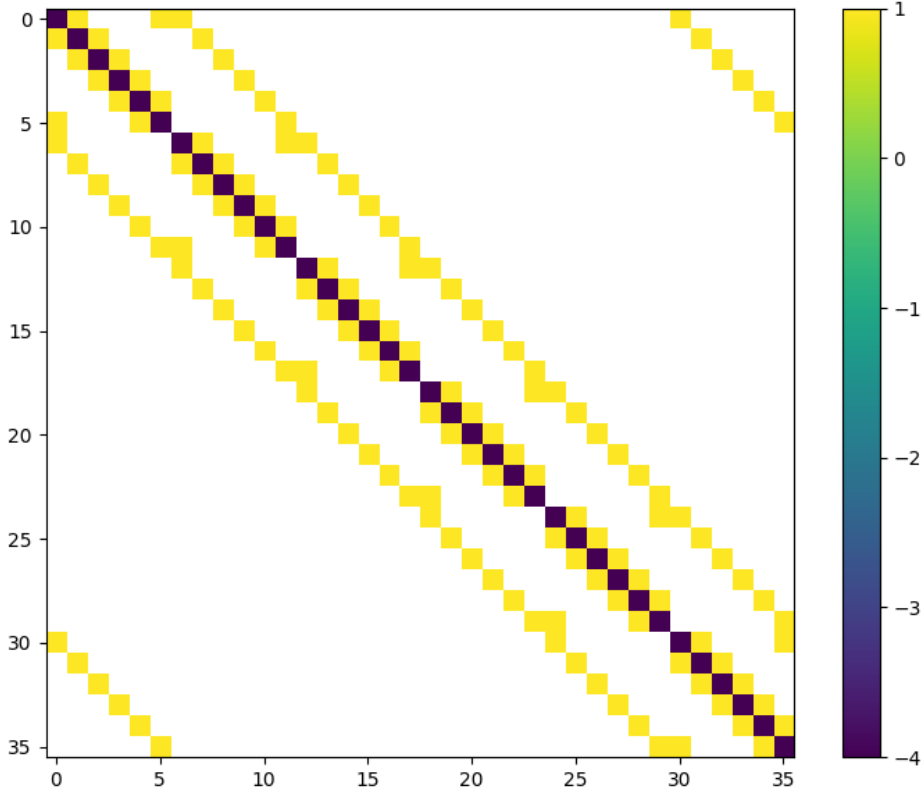


Figure 4: 2-dimensional discrete laplacian operator with discretization of size $N_1 = N_2 = 6$ and discretization step size of $h_1 = h_2 = 1$ for both dimensions show as a heatmap, with zeros removed.

Because we want sets of distinct matrices, as we need a unique set of matrices for the training and a set for the evaluation of the learned preconditioners. Each nonzero value in the matrices will have some random perturbation applied to it. The simplest way to implement this is to perturb the smaller matrices before doing the Kronecker products in equation 46. Because the discretization is the same for both dimensions there is only needed to create three unique matrices to get unique entries for the 2-dimensional laplace operator, see equation (47). To get small perturbation we use the normal distribution with a mean of 1 and small standard deviation. Note that because the perturbations are done to the smaller matrices and the Kronecker product, the final perturbations will have a distribution of $N(1, \sigma^2) \cdot N(1, \sigma^2)$. An example of a perturbed matrix can be seen in figure 5 with an initial perturbation of $N(1, 0.3^2)$.

$$\Delta_p \otimes I_{p_1} + I_{p_2} \otimes \Delta_p. \quad (47)$$

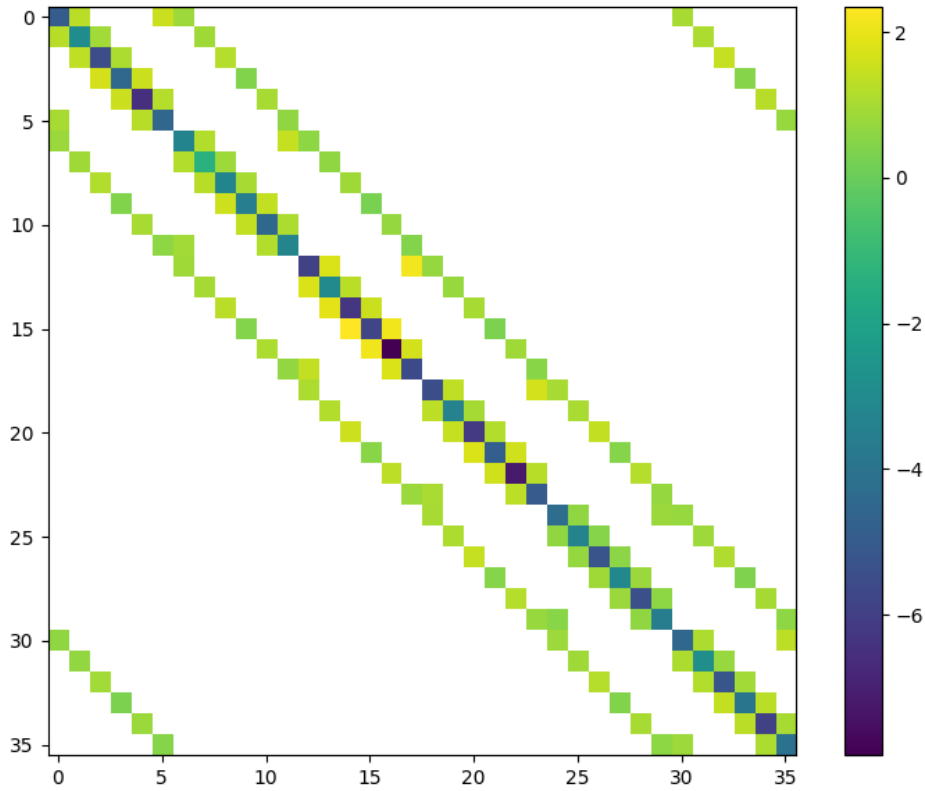


Figure 5: 2-dimensional discrete laplacian operator with discretization of size $N_1 = N_2 = 6$ and discretization step size of $h_1 = h_2 = 1$ for both dimensions with an initial perturbations of $N(1, 0.3^2)$ show as a heatmap, with zeros removed.

Now that we can generate matrices, we need to determine the parameters of the data generation. These parameters are the discretization size of the 1-dimensional laplace operators and the variance of the initial perturbations. What we would like to have for the solver iteration count is that it is larger than the system size, because then the preconditioner has something to work with. On the other hand we don't want the systems to be too ill-conditioned compared to the size of the system, as we do not want breakdowns to occur for the unpreconditioned linear system, or breakdowns in the exploration of the preconditioner. Therefore we will test four different configurations with 250 matrices each set. The configurations can be seen in table

7

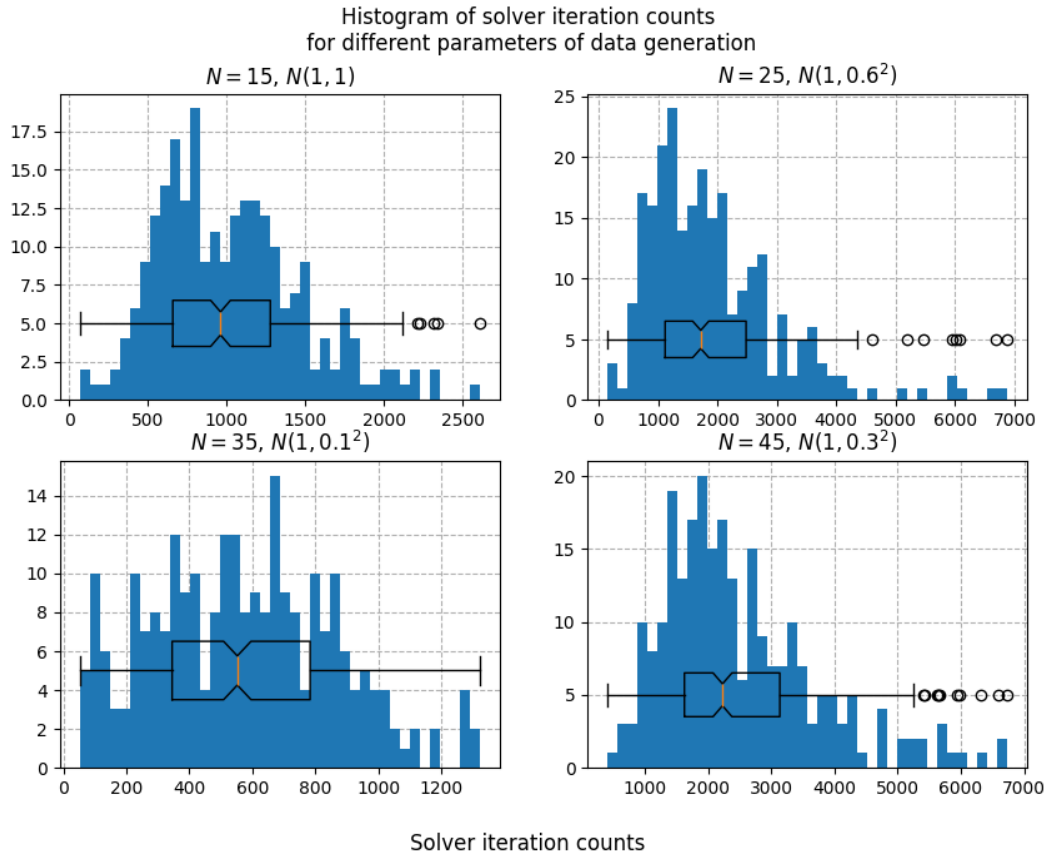


Figure 6: Histograms and boxplot of the solver iteration counts for 4 different configurations of the 2-dimensional discrete laplace operator with perturbations, with breakdown occurrences removed.

N	System Size	Perturbation	Mean	Median	STD	Break downs
15	225	$N(1, 1)$	1019.1	955.5	461.11	17
25	625	$N(1, 0.6^2)$	1927.6	1713	1135.8	2
35	1225	$N(1, 0.1^2)$	567.2	550	290.8	0
45	2025	$N(1, 0.3^2)$	2487.3	2218	1222.4	0

Table 7: Parameter configuration and statistic of the four test runs of the 2-dimensional discrete laplace operator with columns corresponding in order to the size of the 1-dimensional discrete laplace operator; the size of the 2-dimensional discrete laplace operator; the initial perturbation distribution; the mean, median, and standard derivation of the solver iterations; the number of systems where a breakdown occurred.

Histograms of the solver iteration count can be seen in figure 6, and statistic can be seen in table 7. The first thing to notice from these four tests is the effect of the perturbations variance has on the solver iteration count. It has a clear positive correlation, meaning with larger variance, the solve iteration count is increased, to the point of becoming too ill-conditioned compared to the system size. This can be seen for configuration with $N = 15$ and perturbation distribution of $N(1, 1)$. What can not be seen in the histograms, because system that had a breakdown occur are removed for readability of the histogram, is that the breakdown that occurred at the latest happened at solver iteration 67337. This is a problem both because that data point is useless for the analysis of the a potential preconditioner, but also because the solving of each of the systems is done in parallel meaning if one of the system take 6 times as long to solve, each learned step has

to wait on the that system to breakdown or be solved before it can continues with the learning. Then if the perturbations are smaller, as is the case for the configuration $N = 35$ and $N(1, 0.1^2)$, the solver iteration count are also smaller compared to the size of the system, to the point that a potential preconditioner has very few solver iteration count to improve on. For these reason we will go forward with with the last of the configurations with $N = 45$ and $N(1, 0.3^2)$, as the median solver iteration count is larger than the system size, most iteration count are larger than the system size without having any systems with an occurrences of a breakdown.

With the generation of data the procedure for learning and evaluation of the preconditioners needs to be changed. For each of the learning tests two sets of matrices will be generated of equal amount, one set to be used in the learning of the preconditioner and the other for the evaluation of the learned preconditioner. With this we will avoid overfitting the preconditioner, as the learning algorithm did in the two previous sections 3.2 and 3.2.1. In the learning algorithm we need a function to determine when a changed is made to the preconditioner was an improvement or if the changed made the preconditioner worse, this we call the improvement function. The functions will be defined and tested in the next section 3.4. In the evaluation of the learned preconditioner we will look at three different measures to determine if the preconditioner is good. The first two is the mean and median solver iteration count of the preconditioned system, this will compared against mean and median solver iteration count of the linear systems solved without a preconditioner applied. The third measure will count how many individual systems where the preconditioner reduced the solver iteration compared with using no preconditioner.

3.4 Improvement functions

As mentioned in the previous section we need a function to determine if changed made to the preconditioner was an improvement. Because this part of the algorithm is especially important, we will test three different improvement functions, for four different seeds each. The improvement functions will described in the following subsections.

3.4.1 MEAN improvement function

The first improvement function is the MEAN improvement function. It will accept a change made to the preconditioner if the mean solver iteration count of the preconditioned systems is lower than the previous mean solver iteration count.

3.4.2 MEDIAN improvement function

The second improvement function is the MEDIAN improvement function. It will accept a change made to the preconditioner if the median solver iteration count of the preconditioned systems is lower than the previous median solver iteration count.

3.4.3 SIGN improvement function

The last improvement function is the SIGN improvement function. It will accept a change made to the preconditioner if a majority of the preconditioned systems have a decreased solver iteration count compared with solver iteration count using the unchanged preconditioner. The name comes from the implementation where a difference of the individual systems solver iteration counts, then it counts the number of occurrences with negative sign and positive sign.

One problem that might occur with the SIGN improvement function is: on a data set with 100 linear systems. If 55 of the systems have a reduction of 1 solver iteration count with the changed preconditioner, and the last 45 have an increase of 1000 solver iteration counts with the changed preconditioner. This new preconditioner would be accepted by the SIGN improvement function. But with this new preconditioner could have both an increase in mean and median solver iteration count. Therefore would not be accepted by the other two improvement functions. How likely for such a state similar to this example is to occur and if the changes are as big as the given example, is to be determined by the testing of the improvement function.

N	Perturbation	Amount	Base learn steps	Learn Scales	Initial coefficient distribution	Number of partitions
45	$N(1, 0.3^2)$	250	100	$\pm\{1, 0.5, 0.25\}$	$N(0, 0.05^2)$	10

Table 8: Run parameter configuration. The columns corresponds to in order: the size of the 1-dimensional discrete laplace operator; the initial perturbations; the amount of matrices in each of the data sets; the base number of learning iteration; the scales each change will attempt with; the initial coefficient distribution; the number of partition in the preconditioner.

Another point to consider is what to do when change to the preconditioner makes one or more of the systems have a breakdown occur, as this could drastically change the solver iteration count of the system in either direction and skew all three of the improvement functions measure. We could remove the data point from the improvement measure, but this would still change result of the improvement function. Because if system with previous largest iteration count had a breakdown the mean and median would be smaller and for the SIGN improvement function needs to data sets of equal size to make the decision. For these reasons we will also add the condition that all preconditioned systems must converge for a change to the preconditioner to be accepted. This could cause problem if a system starts ill-conditioned or if the initial coefficient of the preconditioner are bad. The new version of the algorithm see algorithm 4.

Algorithm 4 Preconditioner learning algorithm with improvement functions.

Require: A set of matrices and vectors than each form a linear system $A_i x = b_i$

Require: An improvement function f

$M^{-1} \leftarrow$ Generate initial preconditioner

$k_list \leftarrow$ A list of solver iteration count for each $M^{-1} A_i x = M^{-1} b_i$

loop $i = 1, 2, \dots$

loop $s \in \pm\{1, 0.5, 0.25\}$

$M_{new}^{-1} \leftarrow$ Changed M^{-1} with scale s

$k_list_{new} \leftarrow$ A list of solver iteration count for each $M_{new}^{-1} A_i x = M_{new}^{-1} b_i$

if f accepts the new state **and** all systems converged **then**

$k_list \leftarrow k_list_{new}$

$M^{-1} \leftarrow M_{new}^{-1}$

 Break the scale for loop

end if

end loop

end loop

The other run parameters will be the same for all 12 test, see table 8. The choice of the parameters are made from the conclusions from the previous sections tests. With a change to the list of scales $\pm\{1, 0.5, 0.25\}$ as to explore both directions of the possible change to the coefficient. Another change compared to the initial learning in section 3.2 the step of the algorithm that solves the systems with Bi-CGSTab will be done in parallel.

Starting with the MEDIAN improvement function we see from its learning curves in figure 7, that it had hard time finding any changes to the preconditioner that would improve the median solver iteration count, with at most 12 accepted preconditioners for seed 0 and down to 3 accepted for seed 3. Table 9 for all acceptance counts. Because of the few number acceptances the evaluation of the preconditioner on the test matrices set, performs worse in all three measures compared to two other improvement functions for all seeds. Comparing to the unpreconditioned systems the mean and median solver iteration count values are very similar, figure 8, and only about a fourth of the system have a lower iteration count, figure 9. With this we can say that the using MEDIAN improvement function in learning does not work.

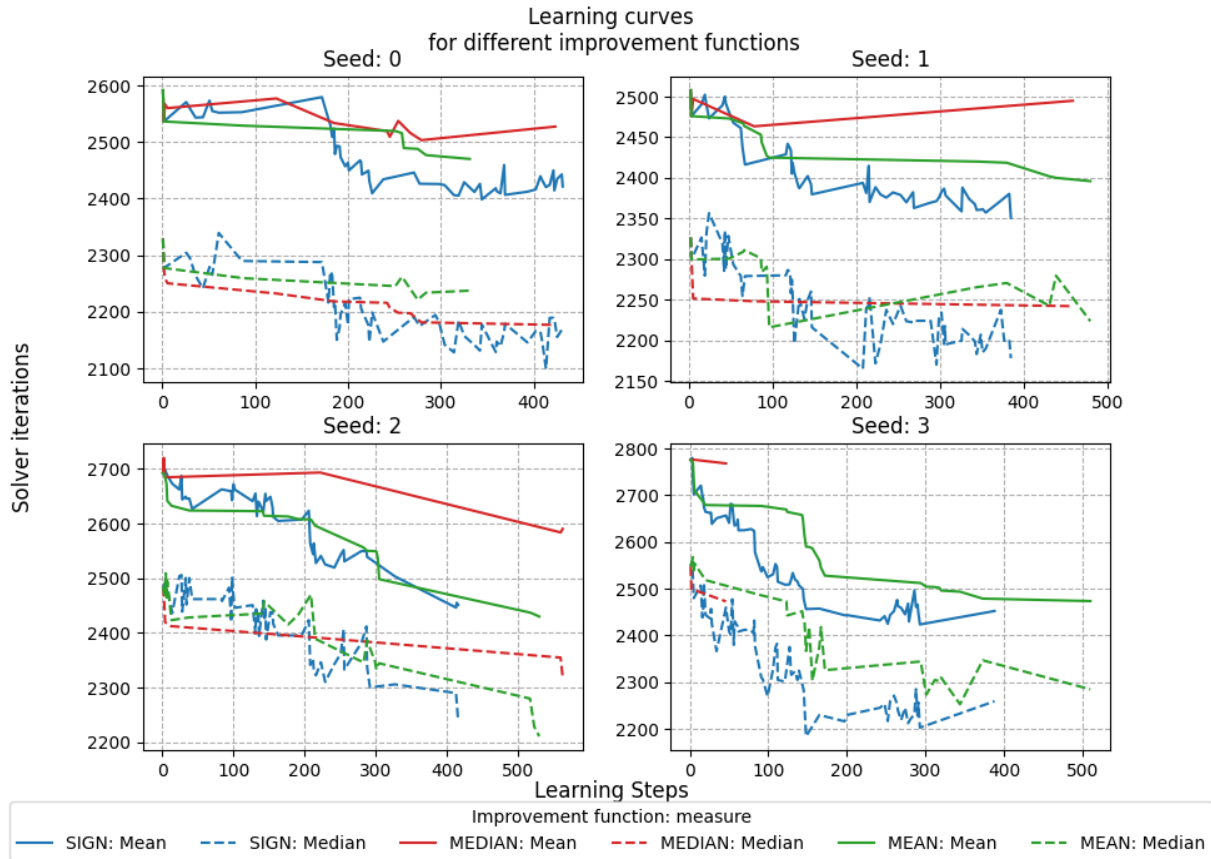


Figure 7: Learning curves for all 12 run configurations. Each plot corresponds to a different seed. The colour of the lines corresponds to a different improvement function: blue for SIGN improvement function; red for MEDIAN improvement function; green for MEAN improvement function. The whole lines is the mean solver iteration count in each learning iteration and the dashed is the median solver iteration count.

With MEAN improvement function the problem of few acceptances is not as prevalent, but still happened for seed 0 with 9 instances of improvement occurring. Of the four runs with MEAN improvement function seed 0 is the one that worst in the evaluations of the learned preconditioner mean and median solver iteration count. While in the individual comparison almost a three fourths of the systems have a lower iteration count compared to the unpreconditioned systems for all four seeds. For the other three seeds the learned preconditioners does decrease the solver iteration counts.

Lastly with SIGN improvement function does not suffer from the problem with few number of improvements, and therefore consistently have a better evaluation of the learned preconditioners. The fluctuations in the learning curves is because of the problem described earlier. From this we can conclude that a preconditioner with this bidiagonal structure can be learned to improve the solver iteration count for an unseen linear systems, with the right improvement function.

Seed	SIGN	MEDIAN	MEAN
0	56	12	9
1	60	5	14
2	52	7	22
3	64	3	25

Table 9: Number of instances a change made to the preconditioner got accepted. A row for each the four seed and columns for each of the three improvement functions.

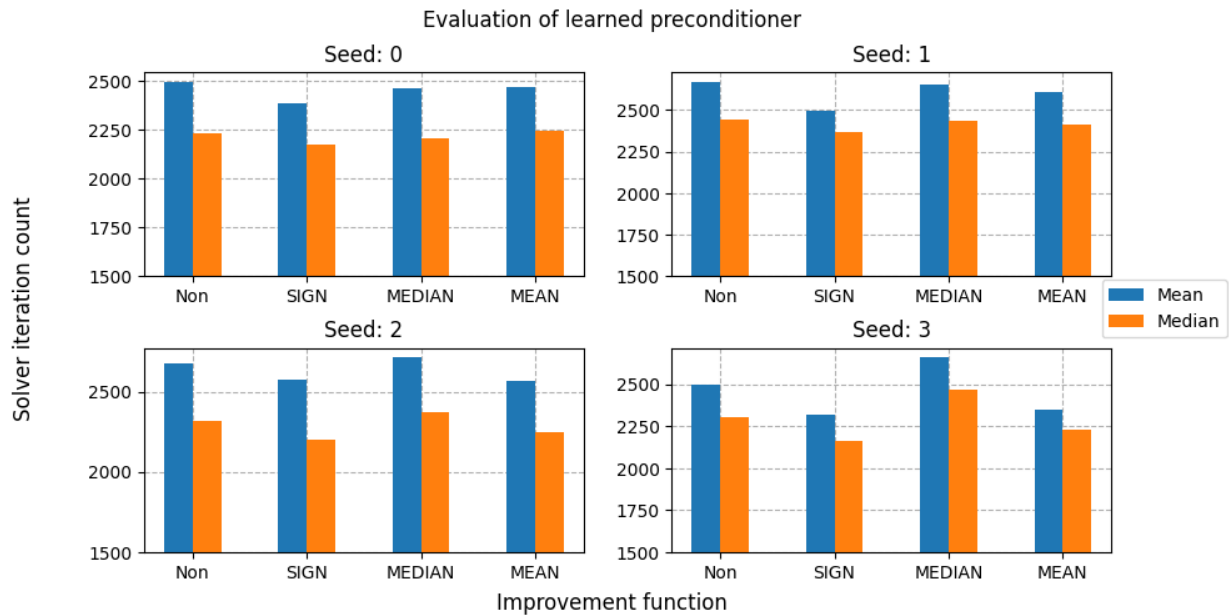


Figure 8: Comparison plot of the evaluation of the learned preconditioner where each plot corresponds to a different seed in the learning. The colour of the bars corresponds to the mean solver iteration count for blue and median solve iteration count for orange. The groups is the different improvement functions used in learning which are in order the unpreconditioned system; SIGN improvement function; MEDIAN as the improvement function; MEAN as the improvement function. Lower is better.

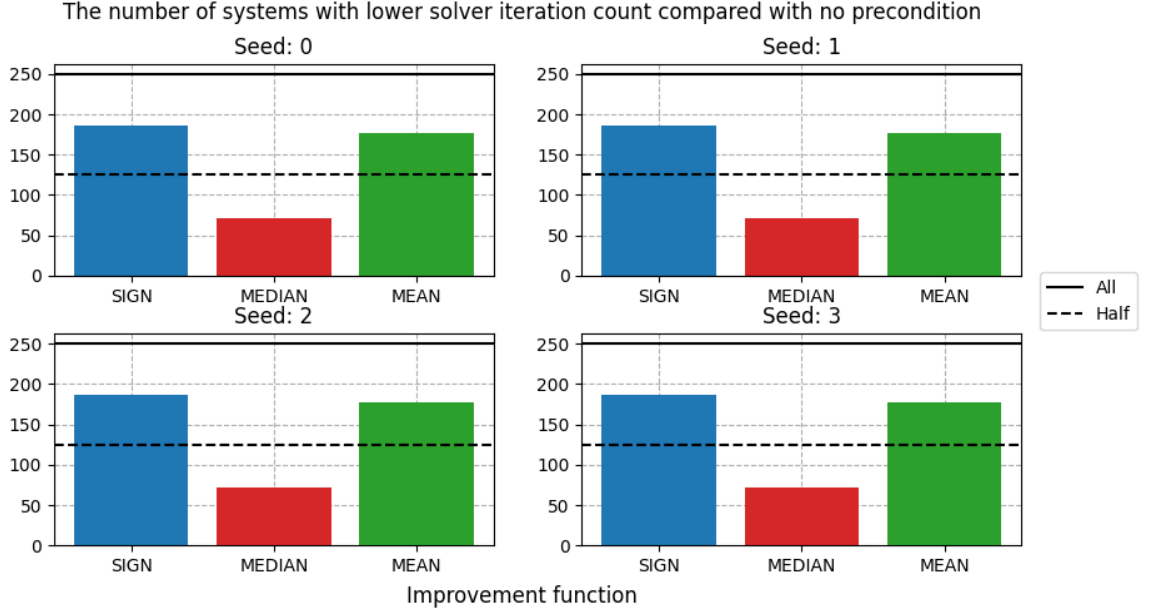


Figure 9: Bar plot of of many of the system where the preconditioner improved the solver iteration count compared to the unpreconditioned systems. Each plot is for a different seed. The bars corresponds to a different improvement function used in learning indicated by its label. The dashed line is half the data set and full is the whole data set. Higher is better.

3.4.4 Comparison with Jacobi preconditioning

For a better comparison of the learned preconditioners, the evaluations should be compared against already know working preconditioner. Because we have information about the structure of the matrices we want to precondition, that is the largest absolute value are on the diagonal and these values do not equal to 1 or 0. Therefore a good choice of preconditioner is the Jacobi preconditioner, as mentioned in section 2.1.2. This preconditioner uses the diagonal elements of a matrix as the preconditioner, $M = \text{diag}(a_{11}, a_{22}, \dots)$ for matrix $A = (a_{i,j})$ and therefore a trivial to compute inverse. Preconditioning the test data with the Jacobi preconditioner and using this set of solver iteration counts as the point of comparison for the learned preconditioner.

What we get from that is the Jacobi preconditioner is better than all learned preconditioners regardless of improvement function and seed, as the mean and median solver iteration count is lower for all four seeds, figure 10. On the individual system level comparison there are systems where the learned preconditioner has lower solver iteration counts, at most 44 systems for seed 1 and SIGN improvement function, figure 11. We conclude from this is that our learned preconditioner can still work, but that the current algorithm or preconditioner needs to be changed.

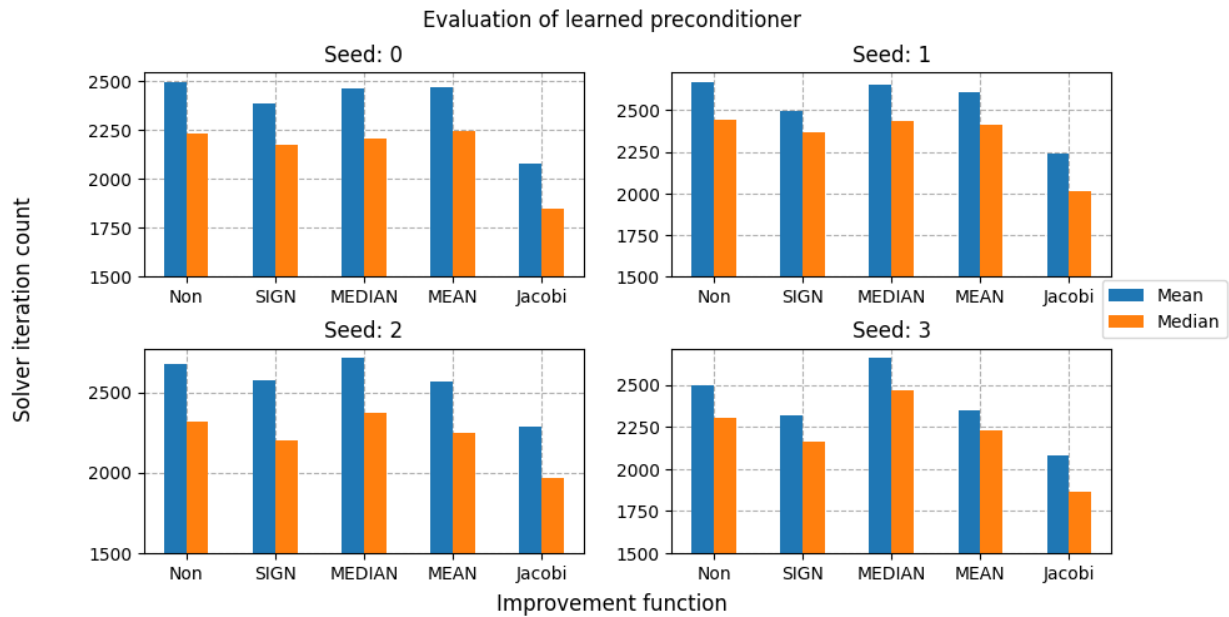


Figure 10: The same plot as figure 8 with a bar group added for the mean and median solver iteration count for the systems preconditioned with the Jacobi preconditioner.

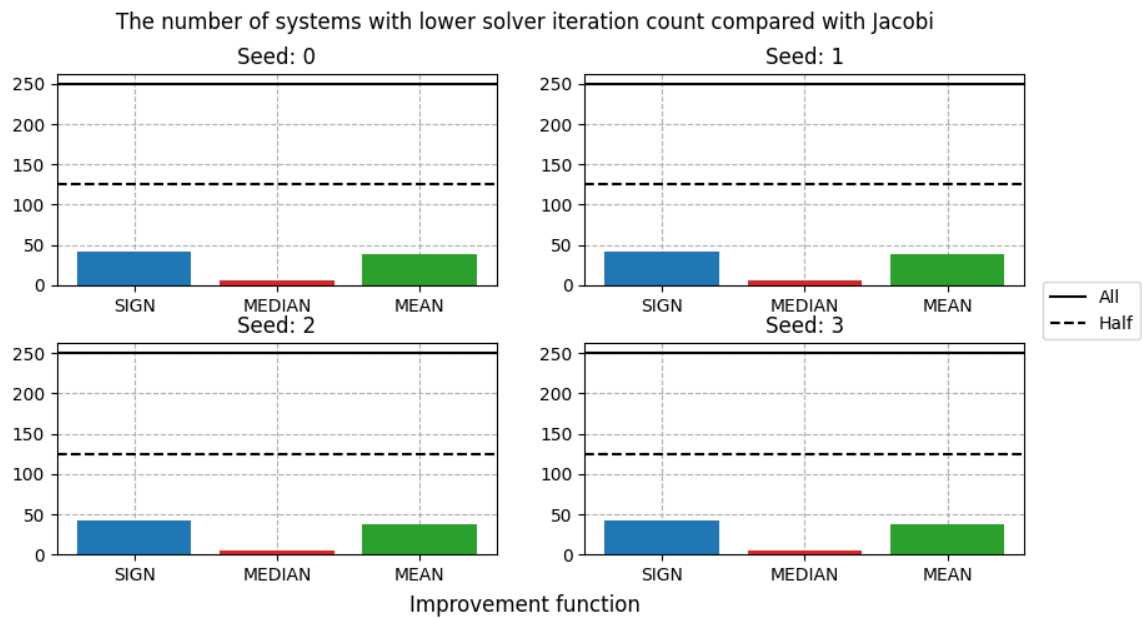


Figure 11: A similar plot to figure 9 with the difference of instead of comparing the solver iteration count to the unpreconditioned system it is compared to the systems preconditioned with Jacobi preconditioner.

3.5 Shifted Jacobi preconditioner

The change we will make is to the structure of the partition shift preconditioner is replacing the diagonal of the matrix from the identity to inverse of the diagonal of the matrix it is applied to. Meaning combining Jacobi preconditioning and our partition shift preconditioner to get a new preconditioner we will call Shifted Jacobi preconditioner. This means our new preconditioner consistent of system specific part, the main diagonal, and a general learned part, the partitioned superdiagonal. A constraint that the new preconditioner has for the matrix it is applied to is that all diagonal entries of the matrix has to have a nonzero value.

To test this new preconditioner we will run the learning algorithm with the same parameters as the previous test in section 3.4 table 8, with the only change being the preconditioner.

Seed	SIGN	MEDIAN	MEAN
0	39	25	29
1	37	27	23
2	58	27	26
3	65	37	34

Table 10: Number of times a new preconditioner was accept. A row for each seed and column for each improvement function.

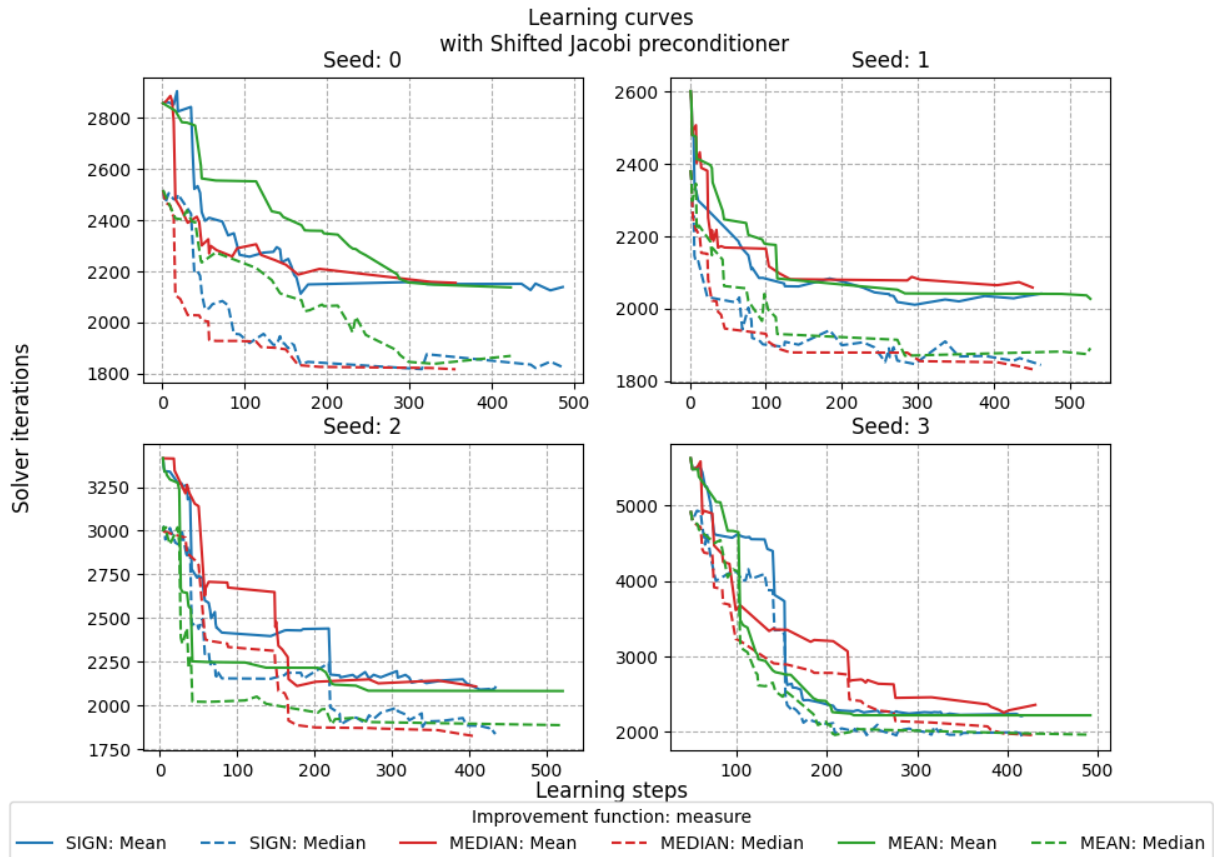


Figure 12: Learning curves for testing Shifted Jacobi preconditioner. The plot has the same structure as figure 7, a plot for each seed, colours indicate improvement function, and line style corresponds to solver iteration count mean or median.

From the learning curves in figure 12, we can see clear improvements when comparing to the learning

curves from the previous tests in figure 7. The mean and median solver iteration count decrease significant amount during the learning for all three improvement function, where as before with MEDIAN improvement function had difficulty finding improvements to the preconditioner. Even when the initial preconditioner starts with mean solver iteration count above 5000 for seed 3 and the final iteration counts is only a bit higher compared to the other seeds, but this may be because the data set and not the preconditioner.

Another improvement is the number of accepted coefficient changes made during learning is considerably larger for MEAN and MEDIAN improvement functions and similar for SIGN, table 10, again comparing to the previous learning tests in section 3.1. This means that even before looking at the evaluation, we can expect the learned preconditioners being good. And lastly a new problem has occurred from the learning curves is that it looks like the learning has plateaued, example for seed 1 the amount of change that happens to the mean and median solver iteration count after learning step 150 is minimal compared to the earlier leaning step. This could mean that it has found the best preconditioner, that it has found a local minimum and the changes made to the preconditioner are not big enough to find a better minimum, or that it did not have enough steps iteration and not plateaued.

In the evaluations of the preconditioner on the test data, there is only a marginal improvement compared to Jacobi preconditioning on its own for the mean and median solver iteration counts, figure 13, and only with SIGN and MEAN improvement functions we get more than half of the systems having a lower solver iteration count than when using Jacobi preconditioning, figure 14. For the differences in improvement function there is very little difference in the evaluations of the learned preconditioners, with SIGN being slightly better across all the seeds for this reason and because it worked the best in the previous test we will only use the SIGN improvement function for all further testing of the learning algorithm and structural changes to the preconditioner. From this it is easy to say that these learned preconditioners with Jacobi diagonal are better than the previous learned preconditioners with identity diagonal, but only slightly better than the simple Jacobi preconditioning.

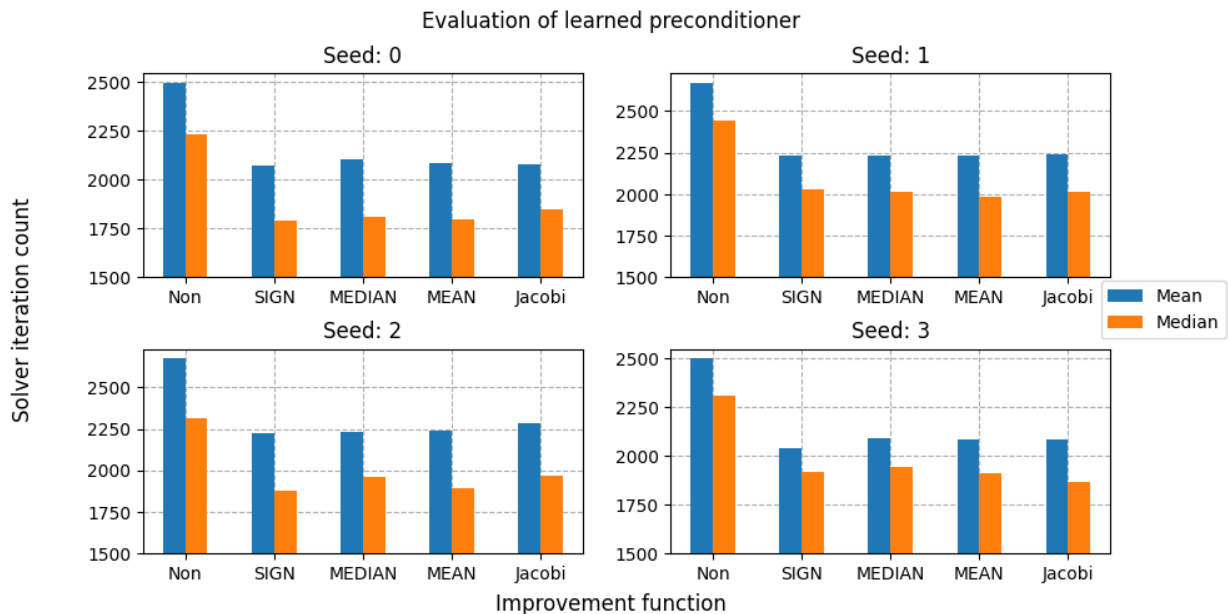


Figure 13: Evaluation of the learned preconditioners with the same structure as figure 10. A plot for each seed, with a bars for the three improvement functions and 2 to compare with unpreconditioned and Jacobi preconditioned solver iteration count. Bar colours indicate mean (blue) and median (orange) solver iteration count.

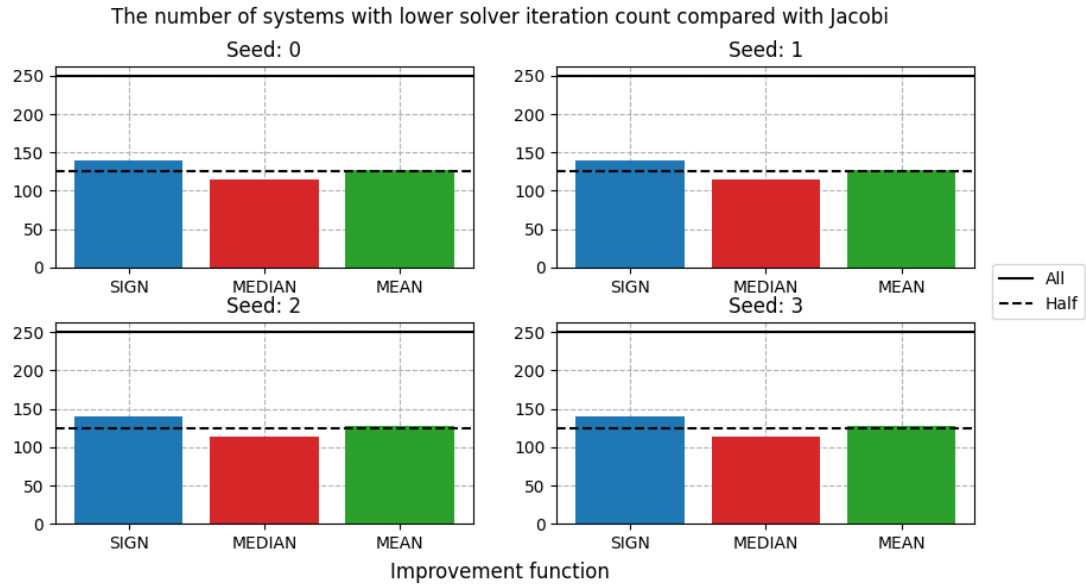


Figure 14: Individual system comparison plot with the same structure as 11. A plot for each seed, bars for each improvement function, and lines to indicate all or half the systems.

3.5.1 Plateau in learning

As mentioned the previous tests learning curves seemed to have plateaued. Therefore we will run the configuration from the previous test with seed 0 and the SIGN improvement function with 500 base learning steps. This is to see if the learning did plateau. If that is the case the solver iteration counts should stay relatively unchanged. Or if the algorithm could find a better preconditioner after the base 100 learning steps from the previous test.

What we get from the learning curve in figure 15 is that the learning has plateaued and no significant changes are made after 500th learning step. The fluctuations are inherent to the SIGN improvement function, which has been mentioned earlier. In the evaluation of the learned preconditioner there is no significant differences between the run with 100 and 500 base learning steps, having only slightly higher mean and median solver iteration counts, this is because in the current version of the algorithm the learned preconditioner is the last version in learning, meaning because of the fluctuations it ended in with a slightly worse preconditioner.

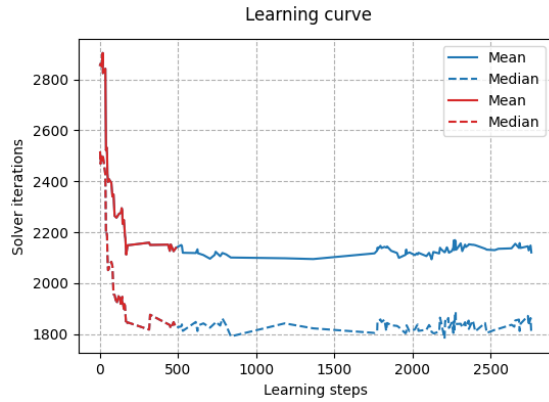


Figure 15: Learning curve for seed 0 and SIGN improvement function with 100 base learning iterations, red curve, and 500 base learning iterations, blue curve. Line style indicate mean (full line) and median (dashed line) solver iteration count.

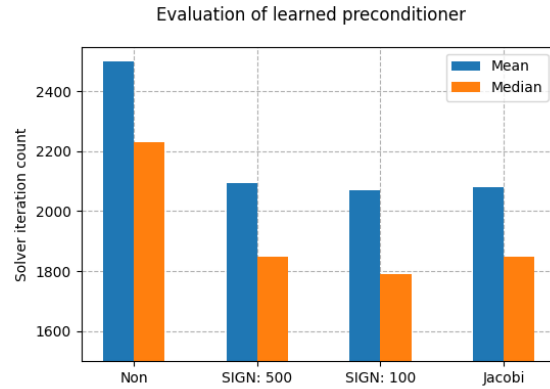


Figure 16: Evaluation of the learned preconditioner, with the same structure as figure 13, for the run with seed 0 and SIGN improvement function with 100 and 500 base learning iterations.

3.6 Allowing disimprovement

To try overcoming the problem of the algorithm plateauing the learning, we will change the learning algorithm such that it can allow changes made to the coefficients to be accepted, even when the improvement function would have decided to rejected the change. We do this because we suspect that the preconditioner is an local minimum, and hope by accepting these changes to find a better minimum. The acceptance of a worse state will be done be with a probability that is inverse proportional to how much worse the change is making the new preconditioner, as to not accepted every change made. Where worse is defined by the improvement function.

Meaning with the SIGN improvement function for a total of 100 matrices: if 45 preconditioned systems solver iteration count decreased and 55 increased it will have a acceptance probability of $\frac{1}{(55-45)+2} = \frac{1}{12}$. The +2 is there for two reasons, first a +1 to ensure no zero division when an equal amount of system have a decrease and increase in solver iteration count, and second another +1 is add such that not all of these changes where this happens will be accepted. The updated pseudocode can be seen in Algorithm 5. This implementation might need adjustment because the probability is also proportional to the total number of matrices. If the data set is small the probability of accepting a state where all system solver iterations count increase would be larger than if the data set was bigger. There is still the condition that all system must converge for any state to be accepted. To test this change to the learning algorithm the same configuration as the runs form the previous section.

From the learning curves in figure 17 and the evaluation of the learned preconditioners in figure 18 and 19, we can see that we gets a slight improvement of the learned preconditioners, meaning we will keep it allowing disimprovements. A thing that was overlooked in the implementation of the disimprovement in the learning algorithm, is that the resulting preconditioner comes from the last accepted learning iteration, meaning that the best found preconditioner is most likely not the preconditioners the algorithm gives as the output. For example if the learning had stopped at solver iteration count spike at learning iteration around 1250, we would have naively concluded that allowing disimprovement is much worse.

Algorithm 5 Preconditioner learning algorithm with with disimprovement**Require:** A set of matrices and vectors than each form a linear system $A_i x = b_i$ **Require:** An improvement function f $M^{-1} \leftarrow$ Generate initial preconditioner $k_list \leftarrow$ A list of solver iteration count for each $M^{-1} A_i x = M^{-1} b_i$ **loop** $i = 1, 2, \dots$ **loop** $s \in \pm \{1, 0.5, 0.25\}$ $M_{new}^{-1} \leftarrow$ Changed M^{-1} with scale s $k_list_{new} \leftarrow$ A list of solver iteration count for each $M_{new}^{-1} A_i x = M_{new}^{-1} b_i$ **if** all systems converged **then****if** f accepts the new state **or** with probability accept a worse state **then** $k_list \leftarrow k_list_{new}$ $M^{-1} \leftarrow M_{new}^{-1}$

Break the scale for loop

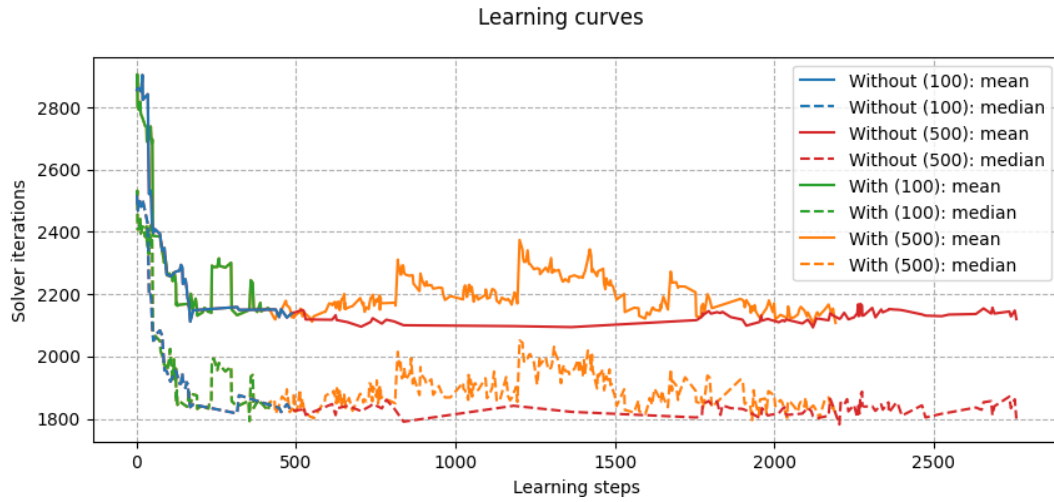
end if**end if****end loop****end loop**

Figure 17: Learning curves for the testing disimprovement where the blue and red lines corresponds to runs without disimprovement with 100 and 500 base learning iterations, this is the same data as in figure 15, green and orange lines are with disimprovement with 100 and 500 base learning iterations, the new test. Line style corresponds to whole line for the mean and dashed line for the median solver iteration count.

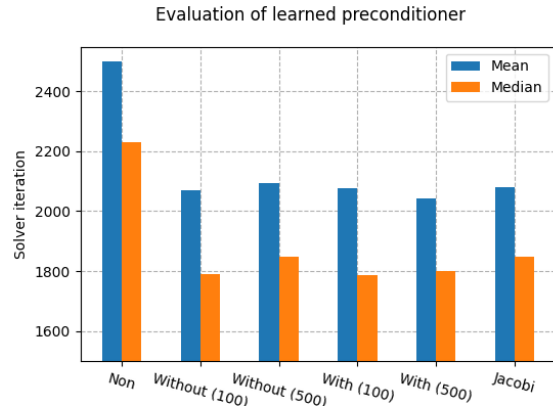


Figure 18: Mean and median solver iteration count for the evaluation of the learned preconditioner. Colour indicate measure: Blue is mean and orange is median solver iteration count. A bar group for each of the four tests: without disimprovement with 100 and 500 base learning iterations and with disimprovement with 100 and 500 base learning iteration. On either end the unpreconditioned and Jacobi preconditioned systems for comparison.

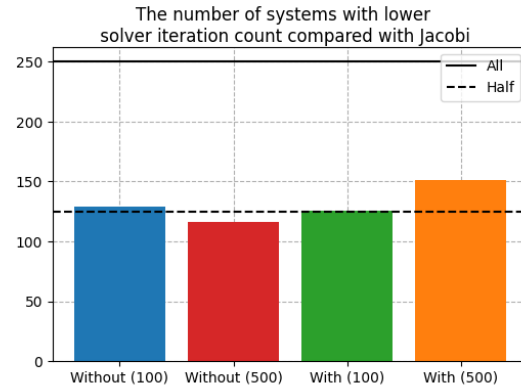


Figure 19: Individual linear system comparison between the learned preconditioners and Jacobi preconditioners with bar size corresponding to the number of systems where the learned preconditioner has a lower solver iteration count that Jacobi Preconditioning.

To resolve this problem the algorithm should keep track of the best seen preconditioner. Here we would like to again use the improvement function as a measure for which were the best, but the SIGN improvement function can not do comparisons across multiple solver iteration count data sets. Therefore we will use the discarded improvement functions mean and median in combination to determine which accepted preconditioner was the best, as these measure are the measures we look at to determine if the learned preconditioner are good in the evaluation. This measure of best will be done after a new preconditioner have been accepted.

3.7 Number of partitions

Because of the results from the previous test, allowing disimprovement, did not improve the learned preconditioner by any significant degree. Therefore we will go back to looking at the structure of our preconditioner. When initial explaining the structure of the Preconditioner, in section 3.1, in the part about the number of partitions of the superdiagonal. We said that with higher number of partitions could lead to a better preconditioner, but with a longer learning time. Therefore the next test will test for larger amount of partitions and the number of partitions we will test is $\{15, 20, 25, 30, 40, 50\}$. To compensate for the potential longer learning time these test will also have an increased base number of learning steps from 100 to 200.

The first thing to notice from the learning curves in figure 20, is that our assumption that having more partitions will equate to longer learning is correct. It can most notable be seen in the test with 50 partition, brown learning curve, as the mean and median solver iteration count does go down but at a slower rate compared to the smaller number of partitions. Starting with a mean solver iteration count of 3086 and decreases it to 2332.3 at learn step 248, but then the solver iteration counts does not change by any substantial amount after that. Compared to the run with 15 partitions gets below a mean solver iteration count 2250 at learn step 138, which means that more partitions have slower learning and it plateaus at higher solver iteration counts.

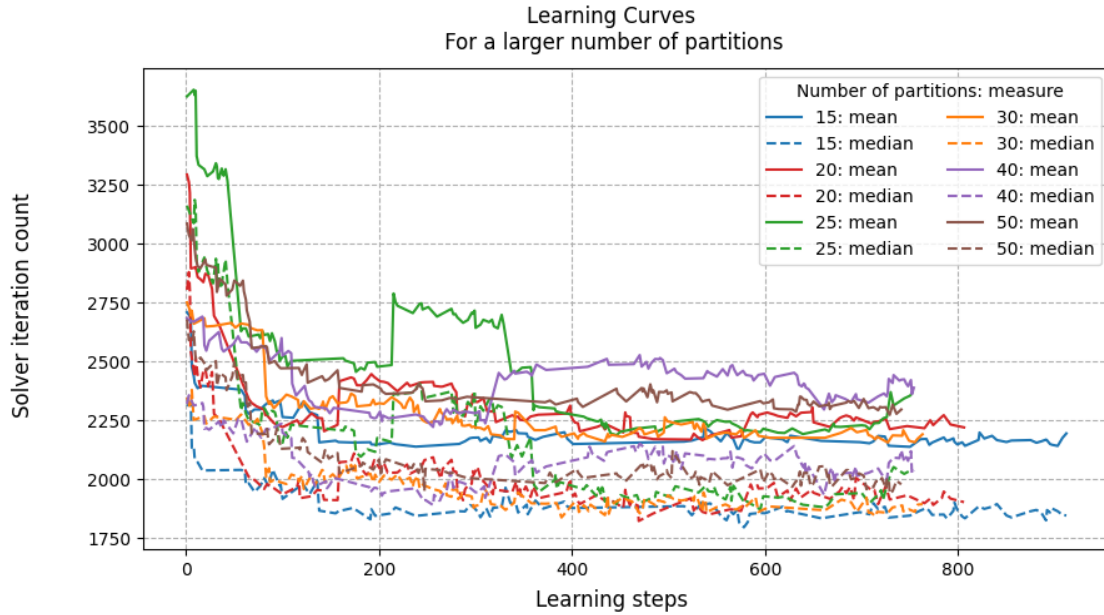


Figure 20: Learning curves for testing larger number of partitions. Where the colour corresponds to the number of partitions and line style corresponds to mean (whole line) and median (dashed line) solver iteration count.

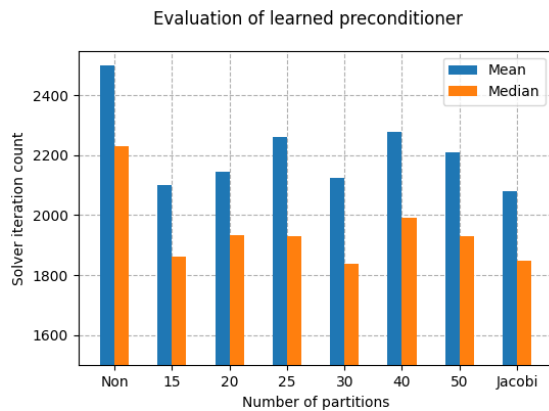


Figure 21: Mean and median solver iteration count indicated by the colour of the bar from the evaluation of the learned preconditioners with different number of partitions indicated by the label.

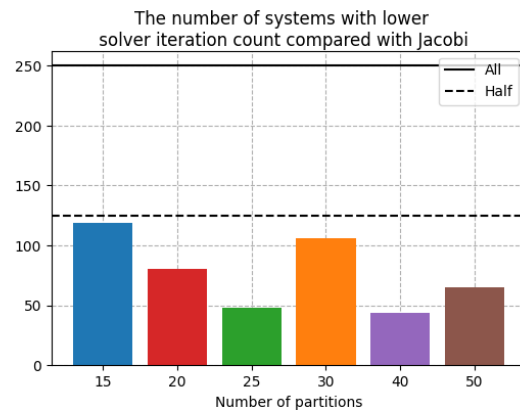


Figure 22: Individual linear system comparison between the learned preconditioners with different number of partitions and Jacobi preconditioners with bar size corresponding to the number of systems where the learned preconditioner has a lower solver iteration count.

The second thing is that allowing accepting of worse state makes the learning worse for higher partition counts, as a consequence of the slower learning. It can most notably be seen for the 40 partition count run, purple learning curve. At learn step 324, the mean solver iteration count increases from 2238.3 to 2424.8, and then it does not get below the increase before it the number of learning steps maxed out. Compare this with the run with 25 partitions, which has increase in solver iteration count at learn iteration 215 and does get back to the iteration from before the increase at learn iteration 337. This problem could be improved with a smaller probability for accepting worse states, but then the learning might get stuck in a minimum.

From the evaluations of the learned Preconditioners, figure 21 and 22, it is clear that using more partitions does not make the preconditioner better. Even with the change to the algorithm such that it returns the best found preconditioner during learning, as both the mean and median solver iteration count are on par, 15 and 30 partition, or worse than using the Jacobi preconditioning. And for all preconditioners the number of systems that have a lower solver iteration count compared to Jacobi is less than half.

3.7.1 Fewer

Because using more partitions did not improvement the learned preconditioners, we will now test with a smaller number of partitions. The partition count we will use in this test is $\{1, 2, 3, 4, 5\}$ and that is the only difference between the runs. With the 200 base learning iterations to make the results comparable with the previous test with more partitions.

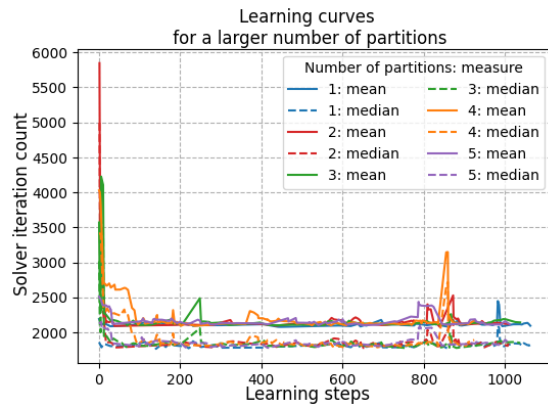


Figure 23: Learning curves for the testing with fewer number of partition, with the same structure as figure 20.

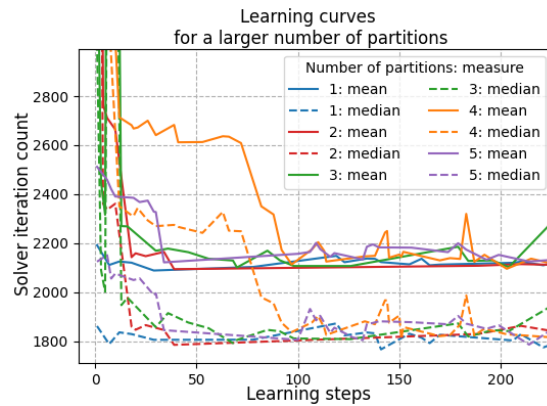


Figure 24: Learning curves for the test with fewer number of partition, the same data as figure 23, but zoomed in to the lower left corner to see the initial learning.

The learning curves, seen in figures 23 and 24, are as expected with fewer partitions has faster learning and it plateaus almost immediately. With the slowest run being with 4 partitions, orange learning curve, plateauing at the 100th learning step. The problem where accepting a worse state and not being able to get back to a similar solver iteration counts does not happen for these runs, as can be seen a bit after the 800th learning step, there is a major spike in solver iteration counts but all tests get back to previous iteration counts at around iteration 900.

In the evaluation of the learned preconditioners, figure 25 and 26, we can see that they are all on par or slightly better than Jacobi preconditioning for all three measure. With using 1 partition begin best in this seed.

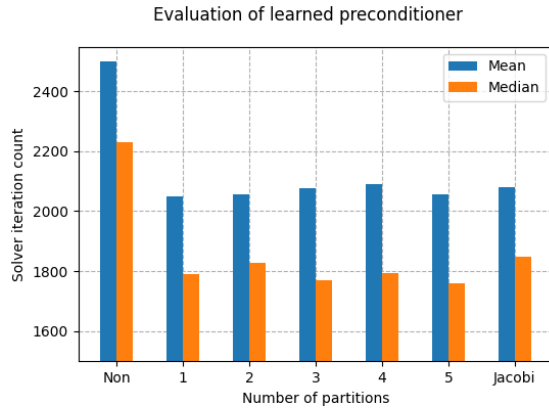


Figure 25: Mean and median solver iteration count from the evaluate of the 5 learned preconditioners. The plot has the same structure as figure 21.

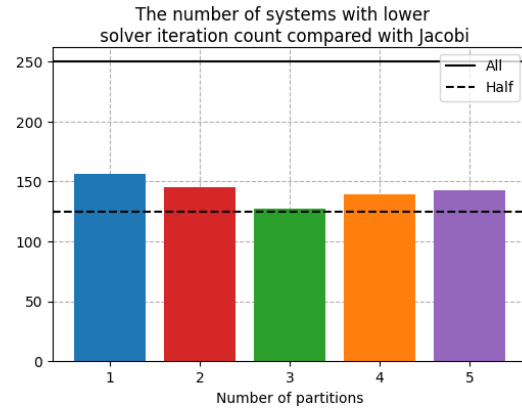


Figure 26: Individual linear system comparison between the learned preconditioners and the Jacobi preconditioner. The plot has the same structure as figure 22.

The coefficients of the learned preconditioners with fewer number partitions, table 11, are not equal, because of the inheritant randomness of the algorithm, but the values are close with an average value of -0.013641246 and standard deviation of 0.00445245 . Because of this and the increased speed in learning. The new standard preconditioner will have 1 partition in the superdiagonal.

Par	Partition coefficients
1	$[-0.01463557]$
2	$[-0.01678746, -0.02122048]$
3	$[-0.00676193, -0.01237617, -0.01208201]$
4	$[-0.02033992, -0.01883074, -0.01200618, -0.01339265]$
5	$[-0.01600863, -0.00575484, -0.00860635, -0.01501886, -0.0107969]$

Table 11: Table of the learned preconditioners coefficients, with a row for each different number of partitions.

From both of the test with changing the number of partitions we did not solve the problem of the preconditioner plateauing, as was the intention with the tests, but we did confirm that using more partitions would result in slower learning, to the point of a good preconditioner being unlearnable, and with fewer learning faster and plateauing quickly.

3.8 Number of diagonals

The next change to the structure we will test is the is the number of superdiagonals with coefficients. The initial reason for using only the first superdiagonal was to keep the preconditioner as spares as possible, to decreases the computational cost of the matrix vector product for the preconditioner. For these runs we will test with $\{1, 2, 3, 4, 5\}$ super diagonals with one partition in each, as to keep the number of coefficients low for faster learning.

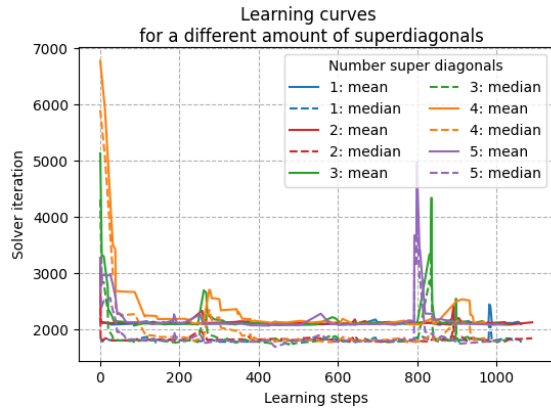


Figure 27: Learning curves for different number of superdiagonals in the preconditioner, with line style indicating mean or median solver iteration counts.

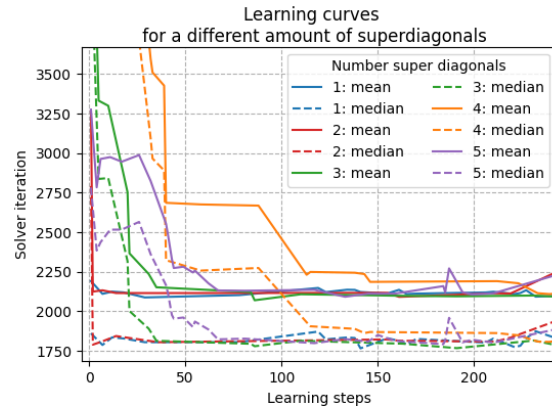


Figure 28: Learning curves for different number of superdiagonals in the preconditioner, same data as figure 27 but zoomed in to the lower left corner to see the initial learning steps.

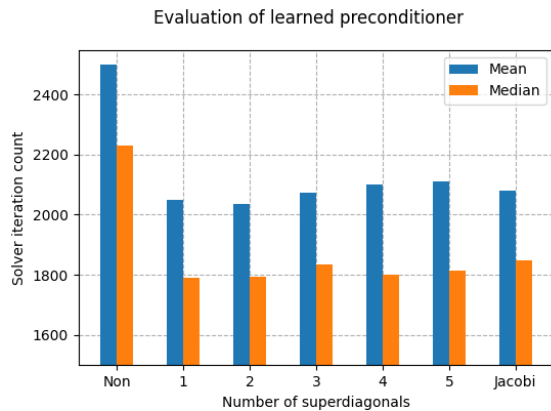


Figure 29: Mean and median solver iteration count from the evaluation of the learned preconditioners with different number of super diagonals, with unpreconditioned and Jacobi preconditioned solver iteration count added for Comparison.

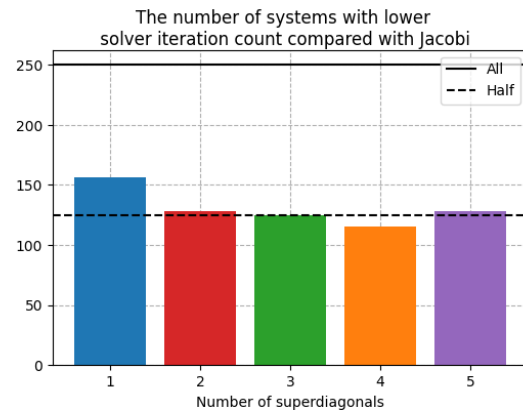


Figure 30: Bar graph showing how many individual preconditioned systems have a lower solver iteration count compared to Jacobi preconditioning.

From the learning curves, figure 27 and 28, we see similar results as the previous test, with regards to the speed of the learning and marginal differences in mean and median solver iteration counts. In the evaluation of the learned preconditioners it is as expected with what was seen in the learning curves. Slight differences in the mean and median solver iteration counts increasing with more superdiagonals, figure 29. And in the individual iteration counts comparison, figure 30, 1 super diagonal clearly preforms the best of the five test. Therefore we conclude that using 1 super diagonal, with no significant difference in evaluation, the faster learning, and most importantly the faster matrix vector product by having fewer nonzero entries.

3.9 Super- and subdiagonals

Because using more superdiagonals did not improve the preconditioner, we will go in the other direction by using both the super- and subdiagonals. We will test using $\{1, 2, 3, 4\}$ super- and subdiagonals with 1 partition in each diagonal, leading to $\{2, 4, 6, 8\}$ different coefficients that needs to be learned.

Immediately from the learning curves in figure 31 seed 0, we see a sizeable improvement. With the plateaued mean solver iteration count around 1750 and median around 1500, compared with the runs of previous section 3.8 having plateaued around mean of 2100 and median of 1800 solver iteration count. Because of the large improvement, the test was run again with a different seed to ensure improvement was not because of the seed of the data set and start conditions of the preconditioners. In the learning curves of this second run we see that the learning curves plateaus at similar solver iteration counts. With the only anomaly being the run with 4 super and 4 subdiagonals, the grey curve. This happened because in implementation the coefficients of the preconditioner is first generated, it then tries to change one coefficients to see if an improvement occurred, but in the first learning iteration there is no point of comparison, meaning the preconditioner is accepted if all preconditioned systems converged and there is the problem. With bad starting coefficients there may not be a change that can be made to the coefficients such that all systems converge. Even with the bad start and the slower learning, because of the 8 coefficients that needs to be learned, its learning curve still gets almost as good results as the runs in this test and a better results compared test from the previous sections.

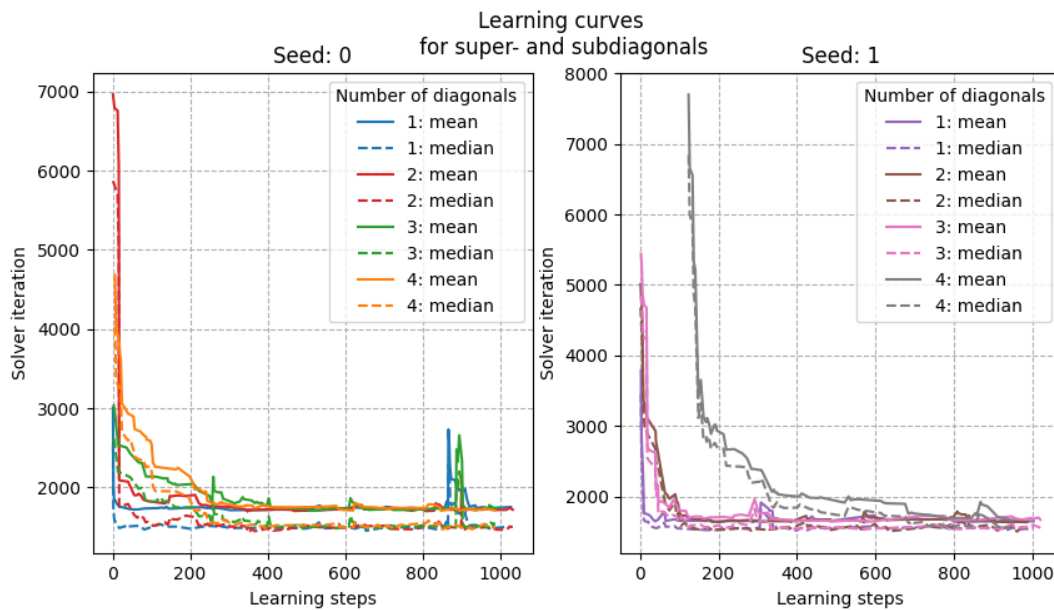


Figure 31: Learning curves for the preconditioner that uses both the super and subdiagonal with a plot for each of the two seeds, with colour corresponding to a different number of super- and subdiagonals and linestyle corresponding to mean or median solver iteration count.

In the evaluation of the learned preconditioners there also is a major improvement. The mean and median solve iteration counts is around 400 iterations fewer compared to Jacobi preconditioning for all diagonal counts and both seeds, figure 32. Where the previous preconditioners with only the superdiagonal was on par with Jacobi preconditioning. In the individual comparison, figure 33, there is a similar improvement, because all preconditioners have more than 240 systems with lower iteration count compared to Jacobi preconditioning, with 1 super- and 1 subdiagonal begin best for both seeds.

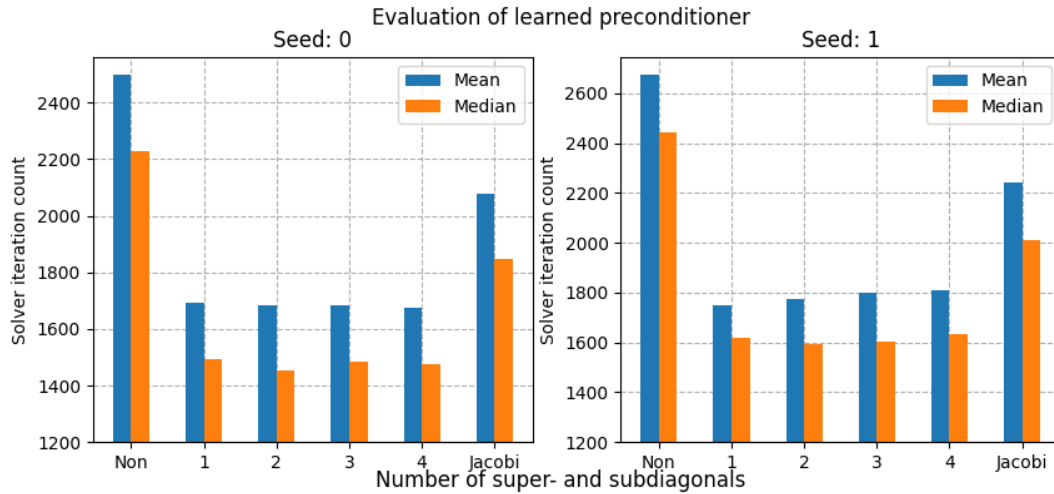


Figure 32: Mean and median solver iteration count for the evaluation of the learned preconditioners with different number of super- and subdiagonals, with a plot for each of the two seeds. With colour corresponding to mean or median solver iteration count.

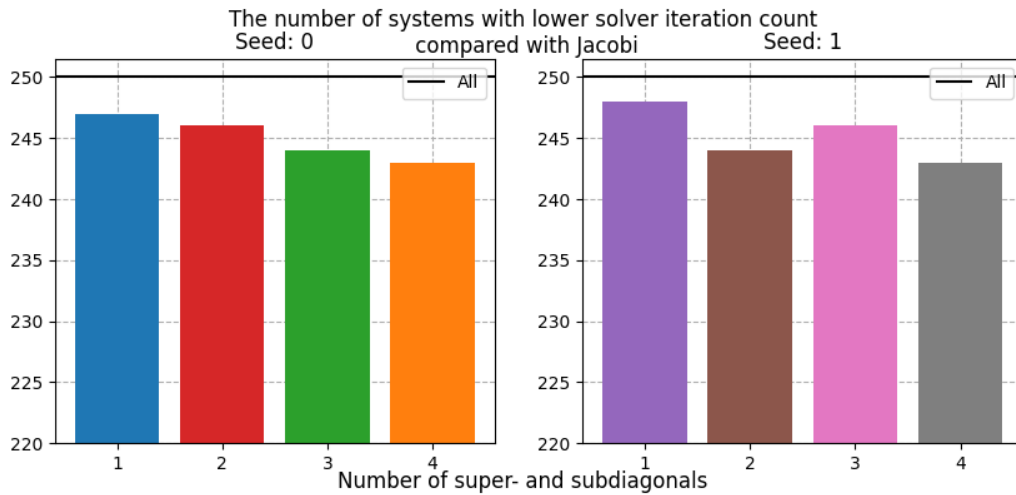


Figure 33: Bar graph showing how many individual systems that has a lower solver iteration count with the learned preconditioners compared with its Jacobi preconditioned counterpart, with a plot for each of the two seeds.

3.9.1 Number of partitions

Because of the significant change made to the preconditioner, we will test if the number of partitions now will have any effect on the learned preconditioners. The number of partitions will be symmetric for the super- and subdiagonal and the number of partitions in the test will be $\{1, 2, 3, 4\}$.

From the learning curves, figure 34, we see similar results as the previous test with different number of partitions in section 3.7. That is more partitions decrease the learning rate with marginal differences in outcome. With one partition count is better in one measure but worse in another. For example with 1 partition has the highest number of individual systems that has a lower solver iteration count compared to Jacobi preconditioned systems, figure 36, but has a worse mean and median solver iteration count compared to the learned preconditioner with 2 partitions.

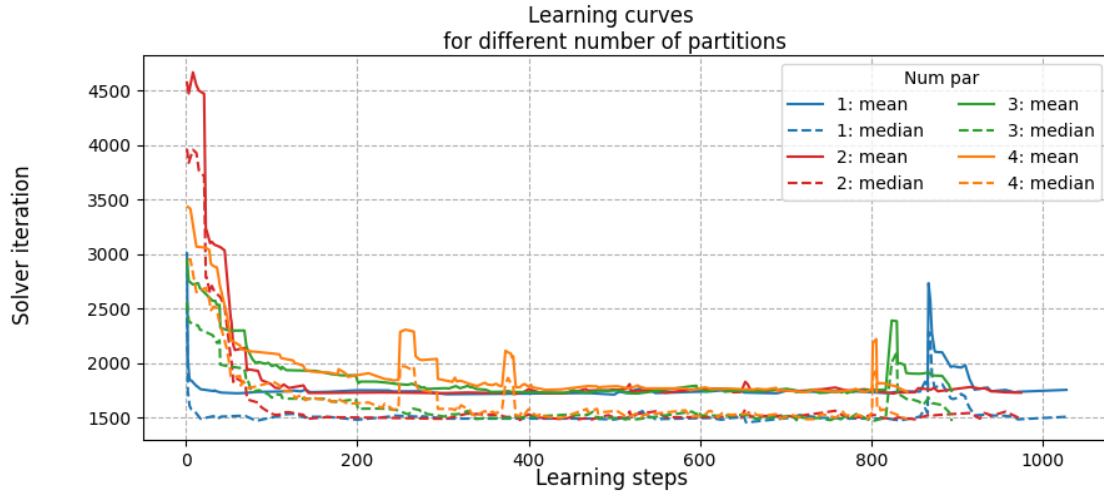


Figure 34: Learning curves for learning preconditioners with 1 super- and subdiagonal. With colour corresponding to a different number of partitions and linestyle the mean or median solver iteration count.

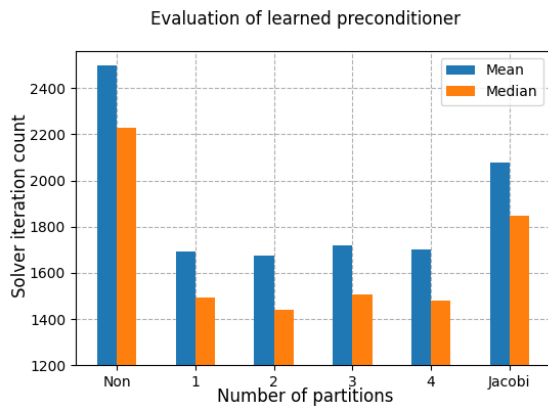


Figure 35: Mean (blue) and median (orange) solver iteration counts from the evaluation of the learned preconditioners, with unpreconditioned and Jacobi preconditioning for comparison.

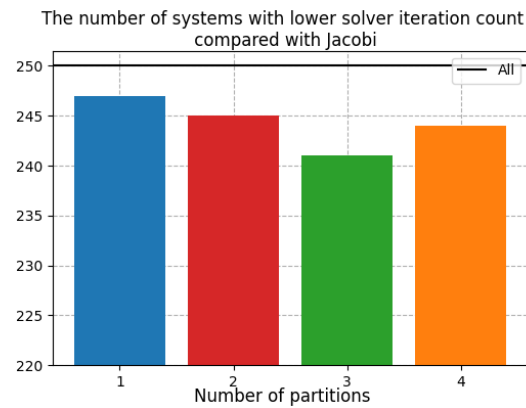


Figure 36: Bar Graph showing how many individual systems that has a lower solver iteration count with the learned preconditioners compared with its Jacobi preconditioned counterpart., With an y-axis offset of 220.

Par	Superdiagonal	Subdiagonal
1	$[-0.059248]$	$[-0.054830]$
2	$[-0.063006, -0.064121]$	$[-0.059046, -0.056173]$
3	$[-0.064320, -0.065399, -0.058531]$	$[-0.055929, -0.074727, -0.055672]$
4	$[-0.046214, -0.058072, -0.047579, -0.063756]$	$[-0.044746, -0.060495, -0.049910, -0.067014]$

Table 12: Table of the learned preconditioners coefficients, with a row for each different number of partitions, columns for the superdiagonal coefficients and one for the subdiagonal coefficients.

For the coefficients of the learned preconditioners, we again see similarities to section 3.7, that being the coefficients are similar, even between the super- and subdiagonal, table 12. Because of this, with only slight differences in learning outcome and the faster learning we can again conclude that using 1 partition each diagonal gets better learned preconditioners.

3.9.2 Symmetry

Because of the similarities of the coefficients between the super- and subdiagonal. We will repeat the test from section 3.9 but with symmetric preconditioners, as this would half the number of coefficients that needs to be learned and therefore faster learning.

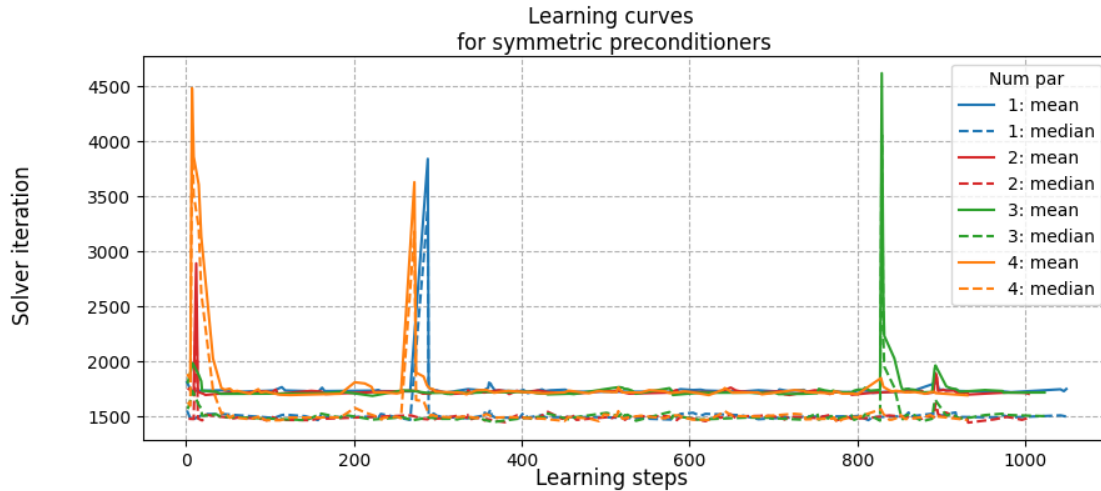


Figure 37: Learning curves for from the learning of symmetric preconditioners with 1 super- and subdiagonal. With colour corresponding to a different number of partitions and linestyle to mean or median solver iteration count.

On major difference in the learning for all of the runs is the actual +run time of each of the configurations. When compared with the similar test of section 3.9 it has increased significantly. The cause of the increase in run time is that changes made to the preconditioners during learning increased the solver iteration counts significantly of most of the systems and therefore some of the learning steps takes significantly longer than other. An example can be seen as the spikes in figure 37 where such a state was accepted. The cause of large increase in solver iteration count because the change made to a coefficient has a larger affect on the preconditioners. A solution would entail a change in the distribution generating the changes made to the coefficients, but changes to this distribution would also have effects on the learning time. With this being the only major difference in the learning of the preconditioner.

In the evaluation of the learned preconditioners, figure 38 and 39, there is only marginal differences between the solver iteration counts of these tests and the test with unsymmetric preconditioners in section 3.9 seed 0. Because of the increase in run time of the learning algorithm we conclude that using a the symmetric variant of our preconditioner bad.

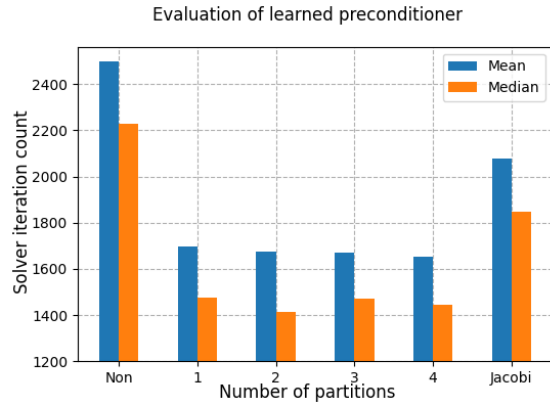


Figure 38: Mean (blue) and median (orange) solver iteration counts from the evaluation of the learned preconditioners, with unpreconditioned and Jacobi preconditioning for comparison.

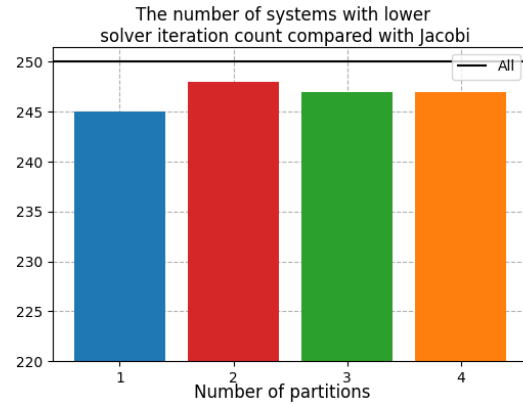


Figure 39: Bar graph showing how many individual systems that has a lower solver iteration count with the learned preconditioners compared with its Jacobi preconditioned counterpart.

3.10 Testing the algorithm on larger systems

With the learning algorithm and structure of the preconditioner gotten to a point where it can reliably learn a preconditioner than can improve the mean solver iteration count by 2000 iterations compared to a trivial preconditioner. There is one parameter that have not changed and that is the size of the 2-dimensional discrete laplace operator used in the training and test data sets. Because no testing has been done on different size laplace operators, importantly larger sizes. The conclusions made from the different tests, especially on the parameter choice and the structure of preconditioner, might be specialized towards the current system size and perturbations. One change that as made to the structure of the preconditioner that is specialized was made in section 3.5. That is the change from an identity diagonal to the Jacobi preconditioner as the diagonal. Because if the matrices in our data sets had diagonals closer to an identity diagonal, Jacobi preconditioning would not make any difference or if the diagonals were close to zero Jacobi preconditioner would have and adverse effect on the preconditioner, and it could therefore struggle to learn a good preconditioner.

Therefore the next test we will change the size of the systems sizes and the perturbations applied to them. Unfortunately this is not as simple as it would seem, because of the condition in our learning algorithm that all systems must converge for a change made to the preconditioner to be accepted. This condition was original implemented because of the problem of comparison between learning iterations where different system does not converge, as it could skew with results of the comparisons in either direction.

To find a suitable larger size and larger perturbations, preliminary testing different size and perturbation distribution data sets on the linear solvers. In these preliminary tests there were almost always at least one or two systems that did not converge. We conclude that it is because of the perturbation distribution being the normal distribution. By having a larger variance and larger system sizes there is more perturbations that is needed to be generated and therefore more likely generate an outlier, that would make the final matrix ill-conditioned. To get more control of outliers we change the distribution that generate the perturbations to a uniform distribution with mean of 1 and variable range, $Uni(1-r, 1+r)$. Testing with this distribution we landed on the 1-dimensional discrete laplacian operator size of 65 resulting in a final system size of 4225 and a range of 0.45 for the perturbations. With these data generation parameters we ran the learning algorithm with 50 base learning steps on 4 different seeds and preconditioner having 1 super- and 1 subdiagonal with 1 partition each.

N	Pertubation distribution	Amount	Base learning steps	Learn Scales	Initial coefficient distribution	Number of partitions	Diagonals
65	$Uni(0.55, 1.45)$	250	50	$\pm\{1, 0.5, 0.25\}$	$N(0, 0.05^2)$	1	$(1, -1)$

Table 13: Table of the run parameter configuration where the columns corresponds to in order: the size of the 1-dimension discrete laplace operator; the initial perturbations on the data; the amount of systems in the data sets; the base number of learning iterations; the scales of the exploration changes; the distribution of the initial coefficients; the number of partitions; the diagonals used.

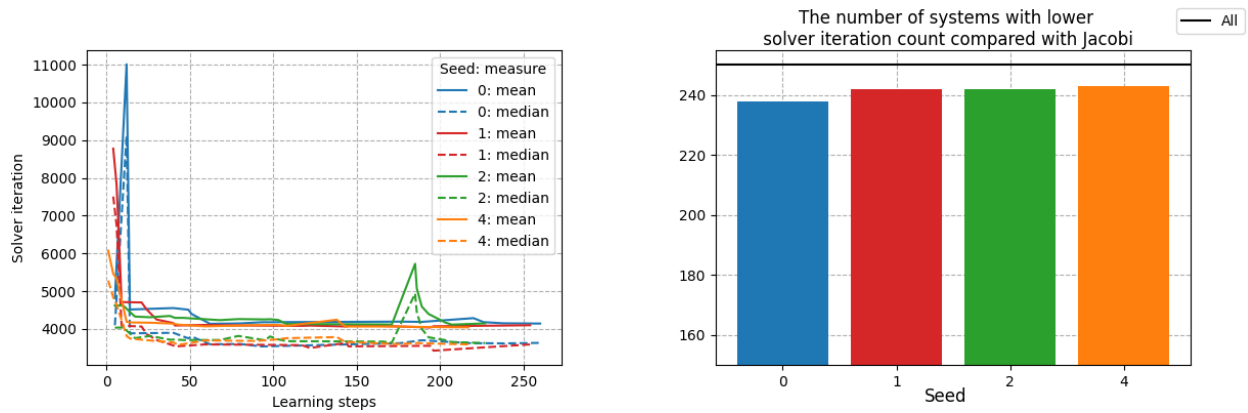


Figure 40: Learning curves for from the learning of the larger systems, where the colour of the curve corresponds to the different seeds.

Figure 41: Graph showing how many individual systems that has a lower solver iteration count with the learned preconditioners compared with its Jacobi preconditioned counterpart from the different seeds. With an y-axis offset of 150.

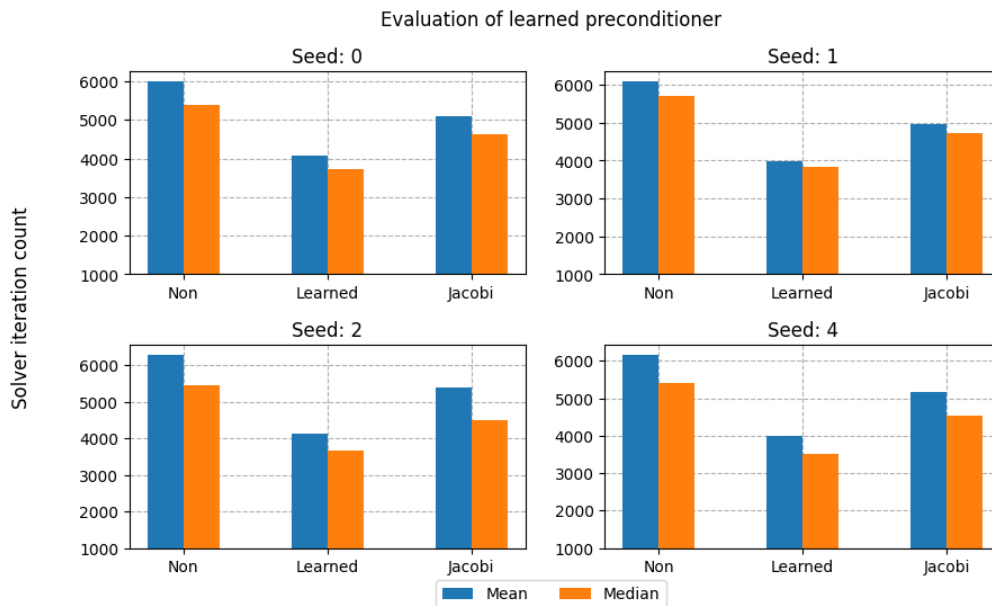


Figure 42: Mean (blue) and median (orange) solver iteration counts from the evaluation of the learned preconditioners, with unpreconditioned and Jacobi preconditioning for comparison, with a plot for each of the seeds.

From learning curves in figure 40 we can see what we have seen many times in the previous tests. That is the mean and median solve iteration counts decrease during the learning quickly and therefore plateaus. For example for seed 1, red curve, starts with a mean solver iteration count 8773.2 and decreases to a count a bit above 4000, and similar curves for the other seeds.

In the evaluation of the learned preconditioners, we can see that the preconditioners decreases the mean and median solver iteration count by approximately 2000 iterations compared to using no preconditioner and 1000 compared with Jacobi preconditioning, figure 42. For the individual comparisons in figure 41 that around 240 system with the learned preconditioner have a lower solver iteration count compared with using Jacobi preconditioning. Meaning it is fair to say that the algorithm has learned a good preconditioner for the larger system sizes for all four seeds.

4 Conclusion

From the exploration we have derived a structure of a preconditioner and a simple learning algorithm that can learn a general part such that it improves the solver iteration count for a set of unseen 2-dimensional discrete laplace operator with perturbations. Therefore we can say that we have accomplished what we set out to do with this project, learn a general preconditioner that decreases the solver iteration counts of unseen linear systems.

With the biggest problem of the current preconditioner is that it might be to overfitted to the structure of the 2-dimensional discrete laplace operator. Meaning that if the current version would try to learn a preconditioner for a matrices with different structure, the learning might not lead to as good results as we have seen from our testings. And would be a future test that would needed to be done.

A part of the problem description that was not done was using an established Machine Learning Algorithm for learning of the preconditioner. The main reason for using our simple learning algorithm, as mentioned, is the need to define a loss or cost function associated to a change made to the potential preconditioner. One problem is the standard loss functions are defined with the expectation of an continues input and because our current measure associated to a preconditioner is the solver iteration counts, it could therefore encounter problems.

5 Bibliography

- [1] Zhaojun Bai et al. *Templates for the Solution of Algebraic Eigenvalue Problems*. Ed. by Zhaojun Bai et al. Society for Industrial and Applied Mathematics, 2000. DOI: 10.1137/1.9780898719581. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9780898719581>. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9780898719581>.
- [2] Richard Barrett et al. “Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods”. In: *Mathematics of Computation* 64 (Sept. 1996). DOI: 10.2307/2153507.
- [3] Alexandre Boutot and Viraj Dsouza. *Explicit Block Encodings of Discrete Laplacians with Mixed Boundary Conditions*. 2026. arXiv: 2603.12405 [quant-ph]. URL: <https://arxiv.org/abs/2603.12405>.
- [4] Y. D. Chong. *Computational Physics*. LibreText. URL: [https://phys.libretexts.org/Bookshelves/Mathematical_Physics_and_Pedagogy/Computational_Physics_\(Chong\)](https://phys.libretexts.org/Bookshelves/Mathematical_Physics_and_Pedagogy/Computational_Physics_(Chong)).
- [5] Nicholas J. Higham. *The Power of Bidiagonal Matrices*. 2023. arXiv: 2311.06609 [math.NA]. URL: <https://arxiv.org/abs/2311.06609>.
- [6] *Matrix market*. URL: <https://math.nist.gov/MatrixMarket/>.
- [7] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Second. Society for Industrial and Applied Mathematics, 2003. DOI: 10.1137/1.9780898718003. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9780898718003>. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9780898718003>.
- [8] Henk Van der Vorst. “Iterative Krylov Methods for Large Linear Systems”. In: 13 (Apr. 2003). DOI: 10.1017/CB09780511615115.

6 Appendix

6.1 Code and data

In the append zip file `code_and_data.zip` is the code, the data used for this project and the Matrix Market matrices used. In the data folder the data is sorted in roughly the the same manner as the section it was shown in. The main code files are: `precondLearn.py` the file that contains the learning algorithm; `Precondition.py` the file for generating and updating the preconditioners; `linearSolver.py` for out implementation of the linear solver; `genLaplace.py` the file that generated the 2-dimensional discrete laplace operators with perturbations; two utility files `util.py` and `genConfig.py`; a subfolder with every file that generated the plots. The rest of the files was made to make some small test.